```
> # ================================================================================
  ======
  # MAPLE TRANSCRIPT: Heisenberg Uncertainty Principle for Cosmological
  Gravitons
  # Wave equation: (R/r)*Laplacian(f) = (1/c^2)*diff(f,t,t)
  # Graviton wave function: psi(r,t) = c2*(r - 2*R)*cos(omega1*t)
  # ================================================================================
  ======
  # Maple Version: 2024 or later
  # All errors corrected:
  #   [1] loglogplot replaced with plot(..., axis[1]={mode=log}, axis[2]=
  {mode=log})
  #   [2] Large float ranges replaced with dimensionless ranges (s=L/R in
  [0,2])
  #   [3] solidsquare  -> solidbox   (valid Maple symbol)
  #   [4] soliddiamondopen -> diamond (valid Maple symbol)
  #   [5] Arrow-function ranges in plot() replaced by explicit variable
  expressions
  #   [6] $ sequence replaced by seq() in textplot loop
  #   [7] Combined display uses sequential list, not Array (avoids 2D/3D
  mixing)
  # ================================================================================
  ======

  restart;
  with(plots):
  with(plottools):
  with(LinearAlgebra):
  with(StringTools):

  # ================================================================================
  ======
  # SECTION 1: PHYSICAL CONSTANTS AND PARAMETERS
  # ================================================================================
  ======

  print("================================================================");
  print(" SECTION 1: PHYSICAL CONSTANTS AND PARAMETERS");
  print("================================================================");

  hbar  := 1.05457e-34;
  c_val := 2.99792458e8;
  G_val := 6.674e-11;
  M_U   := 1.0e53;
  R_val := 3.9228e22;
  GM    := evalf(G_val * M_U);
  c2_val := evalf(2.0*Pi*GM / R_val);
```

```
print("GM     =", GM, "[m^3/s^2]");
print("c2     =", c2_val, "[m^3/s^2]");

# Airy zeros
a1 := -2.33810741;
a2 := -4.08794944;
a3 := -5.52055983;

k1 := evalf(abs(a1)^(3.0/2.0) / (2.0*sqrt(2.0)*R_val));
k2 := evalf(abs(a2)^(3.0/2.0) / (2.0*sqrt(2.0)*R_val));
k3 := evalf(abs(a3)^(3.0/2.0) / (2.0*sqrt(2.0)*R_val));

print("k1 =", k1, "[m^-1]");
print("k2 =", k2, "[m^-1]");
print("k3 =", k3, "[m^-1]");

omega1 := evalf(k1 * c_val);
T1     := evalf(2.0*Pi / omega1);
E1     := evalf(hbar * omega1);
m_g    := evalf(E1 / c_val^2);

print("omega1 =", omega1, "[rad/s]");
print("T1     =", T1, "[s] =", evalf(T1/3.156e13), "[Myr]");
print("E1     =", E1, "[J]");
print("m_g   <=", m_g, "[kg]");

# ================================================================================
======
# SECTION 2: NORMALIZATION
# ================================================================================
======

print("================================================================");
print(" SECTION 2: NORMALIZATION");
print("================================================================");

N_norm := evalf(sqrt(3.0 / (8.0*R_val^3)));
print("N =", N_norm, "[m^(-3/2)]");
print("N^2 * (8R^3/3) =", evalf(N_norm^2 * 8.0*R_val^3/3.0));  # = 1

# ================================================================================
======
# SECTION 3: POSITION EXPECTATION VALUES
# ================================================================================
======

print("================================================================");
print(" SECTION 3: POSITION EXPECTATION VALUES");
print("================================================================");
```

```
r_mean  := evalf(R_val / 2.0);
r2_mean := evalf(2.0*R_val^2 / 5.0);
var_r   := evalf(r2_mean - r_mean^2);
Delta_r := evalf(sqrt(var_r));

print("<r>      =", r_mean, "[m]  ( = R/2 )");
print("<r^2>    =", r2_mean, "[m^2]  ( = 2R^2/5 )");
print("Delta_r  =", Delta_r, "[m]  ( = R*sqrt(3/20) )");
print("Delta_r/R =", evalf(Delta_r/R_val));   # 0.3873

# ================================================================================
======
# SECTION 4: MOMENTUM EXPECTATION VALUES
# ================================================================================
======

print("================================================================");
print(" SECTION 4: MOMENTUM EXPECTATION VALUES");
print("================================================================");

p2_mean := evalf(hbar^2 * k1^2 / 4.0);
Delta_p := evalf(sqrt(p2_mean));

print("<p>      = 0  [kg*m/s]  (real wave function)");
print("Delta_p  =", Delta_p, "[kg*m/s]");

# ================================================================================
======
# SECTION 5: HEISENBERG PRODUCTS
# ================================================================================
======

print("================================================================");
print(" SECTION 5: HEISENBERG UNCERTAINTY PRODUCTS");
print("================================================================");

UP_pos_mom := evalf(Delta_r * Delta_p);
hbar2      := evalf(hbar / 2.0);
ratio_pos  := evalf(UP_pos_mom / hbar2);
Delta_E    := evalf(hbar * omega1 / 2.0);
Delta_t    := evalf(1.0 / omega1);
UP_E_t     := evalf(Delta_E * Delta_t);

print("Delta_r * Delta_p     =", UP_pos_mom, "[J*s]");
print("hbar/2                =", hbar2,      "[J*s]");
print("Ratio (pos-mom)       =", ratio_pos);
print("Delta_E               =", Delta_E, "[J]");
print("Delta_t               =", Delta_t, "[s] =", evalf
(Delta_t/3.156e13), "[Myr]");
```

```
print("Delta_E * Delta_t     =", UP_E_t, "[J*s]");
print("Ratio (E-t)           =", evalf(UP_E_t/hbar2));   # = 1

# Coherent and localization widths
sigma_r     := evalf(sqrt(hbar / (2.0*m_g*omega1)));
delta_r_min := evalf(hbar * c_val / E1);

print("sigma_r (coherent)    =", sigma_r, "[m]  (=", evalf(sigma_r/R_val),
"R)");
print("delta_r_min           =", delta_r_min, "[m]  (=", evalf
(delta_r_min/R_val), "R)");

# Force uncertainty
c2n          := c2_val;
dFdr_mean    := evalf(c2n/(4.0*Pi) * (4.0*R_val - r_mean) / r_mean^3);
Delta_F      := evalf(abs(dFdr_mean) * Delta_r);
print("Delta_F               =", Delta_F, "[N/kg]");

# ================================================================================
======
# SECTION 6: COSMOLOGICAL CONSTANT
# ================================================================================
======

print("================================================================");
print(" SECTION 6: COSMOLOGICAL CONSTANT PROBLEM");
print("================================================================");

V_Universe   := evalf((4.0*Pi/3.0) * (2.0*R_val)^3);
rho_grav_vac := evalf(E1 / (2.0 * V_Universe));
rho_Lobs     := 6.0e-10;

print("rho_grav_vac  =", rho_grav_vac, "[J/m^3]");
print("rho_Lambda    =", rho_Lobs,     "[J/m^3]");
print("Discrepancy   =", evalf(rho_Lobs/rho_grav_vac));

# ================================================================================
======
# PLOT 1: Wave Function and Probability Density
# ================================================================================
======

print("================================================================");
print(" PLOT 1: Wave Function and Probability Density");
print("================================================================");

p1a := plot(sqrt(3.0/8.0)*(x-2.0), x=0..2,
    color="DodgerBlue", thickness=3, linestyle=solid,
    legend="psi(r/R)  normalized");
```

```
p1b := plot((3.0/8.0)*(x-2.0)^2, x=0..2,
    color="OrangeRed", thickness=3, linestyle=dash,
    legend="|psi|^2 = P(r/R)");

Delta_r_sc := evalf(Delta_r/R_val);
r_mean_sc  := evalf(r_mean/R_val);

p1c := plots[display](
    plottools[line]([0.5,-0.5],[0.5,0.75],
        color="ForestGreen", thickness=2, linestyle=dot));

p1d := plots[display](
    plottools[line]([r_mean_sc-Delta_r_sc, 0.0],
                    [r_mean_sc-Delta_r_sc, 0.45],
        color="Purple", thickness=2, linestyle=dashdot),
    plottools[line]([r_mean_sc+Delta_r_sc, 0.0],
                    [r_mean_sc+Delta_r_sc, 0.45],
        color="Purple", thickness=2, linestyle=dashdot));

PLOT1 := plots[display](p1a, p1b, p1c, p1d,
    title="Graviton Wave Function and Probability Density\npsi(r)=N*
(r-2R),  x=r/R",
    axes=boxed, gridlines=true,
    labels=["r/R","psi  or  |psi|^2"],
    labeldirections=[horizontal,vertical],
    size=[800,500]);

print("Plot 1 done."); PLOT1;

# ================================================================================
======
# PLOT 2: Airy Eigenstates
# ================================================================================
======

print("============================================================");
print(" PLOT 2: Airy Function Eigenstates g_n(r/R)");
print("============================================================");

eps := 0.02;

g1v := [seq(evalf(abs(AiryAi(-abs(a1)*(0.01+i*0.01)/2.0)/(0.01+i*0.01)
/2.0+eps)), i=1..199)];
g2v := [seq(evalf(abs(AiryAi(-abs(a2)*(0.01+i*0.01)/2.0)/(0.01+i*0.01)
/2.0+eps)), i=1..199)];
g3v := [seq(evalf(abs(AiryAi(-abs(a3)*(0.01+i*0.01)/2.0)/(0.01+i*0.01)
/2.0+eps)), i=1..199)];

g1mx := max(op(g1v)); g2mx := max(op(g2v)); g3mx := max(op(g3v));
```

```
p2a := plot(AiryAi(-abs(a1)*x/2.0)/(x/2.0+eps)/g1mx, x=0.05..2,
    color="DodgerBlue", thickness=3, legend="n=1 fundamental");
p2b := plot(AiryAi(-abs(a2)*x/2.0)/(x/2.0+eps)/g2mx, x=0.05..2,
    color="OrangeRed", thickness=2, linestyle=dash, legend="n=2 first
overtone");
p2c := plot(AiryAi(-abs(a3)*x/2.0)/(x/2.0+eps)/g3mx, x=0.05..2,
    color="ForestGreen", thickness=2, linestyle=dashdot, legend="n=3
second overtone");
p2d := plot(0.0, x=0..2, color="Gray", thickness=1);

PLOT2 := plots[display](p2a, p2b, p2c, p2d,
    title="Airy Eigenstates g_n(r)  of  (R/r)*Laplacian(g)=-k_n^2*g\ng(2R)
=0, peak-normalized",
    axes=boxed, gridlines=true,
    labels=["r/R","g_n  [normalized]"],
    labeldirections=[horizontal,vertical],
    size=[800,500]);

print("Plot 2 done."); PLOT2;

# ================================================================================
======
# PLOT 3: Force comparison — log-log using axis[n]=[mode=log]
# FIX: plot() with axis option, NOT loglogplot()
# ================================================================================
======

print("================================================================");
print(" PLOT 3: Modified Force vs Newton (log-log)");
print("================================================================");

GM_R2 := evalf(GM / R_val^2);

# |F_mod| and |F_Newton| in units of GM/R^2, x=r/R, attractive regime x<2
p3a := plot(GM_R2*abs(x-2.0)/(2.0*x^2),  x=0.001..1.9,
    color="DodgerBlue", thickness=3,
    axis[1]=[mode=log], axis[2]=[mode=log],
    legend="|F_modified|*R^2/GM");

p3b := plot(GM_R2/x^2, x=0.001..1.9,
    color="OrangeRed", thickness=2, linestyle=dash,
    axis[1]=[mode=log], axis[2]=[mode=log],
    legend="|F_Newton|*R^2/GM = 1/x^2");

p3c := plot(abs(abs(x-2.0)/2.0 - 1.0), x=0.001..1.9,
    color="Purple", thickness=2, linestyle=dot,
    axis[1]=[mode=log], axis[2]=[mode=log],
    legend="Relative deviation |F_mod-F_N|/|F_N|");
```

```
PLOT3 := plots[display](p3a, p3b, p3c,
    title="Modified Gravitational Force vs Newton  (log-log)\nF(r)=GM/(2R)
*(1/r-2R/r^2),  x=r/R",
    axes=boxed, gridlines=true,
    labels=["r/R  [log]","|Force|*R^2/GM  [log]"],
    labeldirections=[horizontal,vertical],
    size=[800,500]);

print("Plot 3 done."); PLOT3;

# ================================================================
======
# PLOT 4: Delta_r and <r> vs boundary — dimensionless variable s=L/R in
[0,2]
# FIX: Use s=0..2 (integer endpoints), not L=0..2*R_val (large float)
# ================================================================
======

print("============================================================");
print(" PLOT 4: Delta_r and <r> vs Boundary s=L/R");
print("============================================================");

# s = L/R:  Delta_r/R = s*sqrt(3/20)/2,   <r>/R = s/4
p4a := plot(s*sqrt(3.0/20.0)/2.0, s=0..2,
    color="DodgerBlue", thickness=3,
    legend="Delta_r(L)/R");
p4b := plot(s/4.0, s=0..2,
    color="OrangeRed", thickness=2, linestyle=dash,
    legend="<r>(L)/R");
p4c := plots[display](
    plottools[line]([2.0,0.0],[2.0,0.5],
        color="Gray", thickness=2, linestyle=dot));

PLOT4 := plots[display](p4a, p4b, p4c,
    title="Position Mean and Uncertainty vs Boundary s=L/R\npsi(r)=N*(r-
L), physical: s=2 (L=2R)",
    axes=boxed, gridlines=true,
    labels=["s = L/R","Position (units of R)"],
    labeldirections=[horizontal,vertical],
    view=[0..2, 0..0.6],
    size=[800,500]);

print("Plot 4 done."); PLOT4;

# ================================================================
======
# PLOT 5: Momentum space distribution — use proc, plot with quoted
expression
# FIX: Define psi_hat_mod2 as a proc; call plot with quoted proc call
```

```
# ================================================================================
======

print("================================================================");
print(" PLOT 5: Momentum Space |psi_hat(k)|^2");
print("================================================================");

Nv   := N_norm;
Rv   := R_val;
k1v  := k1;

psi_hat_mod2 := proc(kappa::numeric)
    local kk, A, B;
    kk := kappa * k1v;
    A  := Nv * ((1.0-cos(2.0*kk*Rv))/kk^2 - 2.0*Rv*sin(2.0*kk*Rv)/kk);
    B  := Nv * (sin(2.0*kk*Rv)/kk^2 + 2.0*Rv*(1.0-cos(2.0*kk*Rv))/kk);
    evalf(A^2 + B^2);
end proc:

peak05 := psi_hat_mod2(0.05):

p5a := plot('psi_hat_mod2(kappa)/peak05', kappa=0.05..8.0,
    color="DodgerBlue", thickness=3, numpoints=300,
    legend="|psi_hat(k)|^2  (norm. to kappa=0.05)");

p5b := plots[display](
    plottools[line]([1.0,0.0],[1.0,1.1],
        color="OrangeRed", thickness=2, linestyle=dot));
p5c := plots[display](
    plottools[line]([evalf(k2/k1),0.0],[evalf(k2/k1),0.6],
        color="ForestGreen", thickness=2, linestyle=dashdot),
    plottools[line]([evalf(k3/k1),0.0],[evalf(k3/k1),0.4],
        color="Purple", thickness=2, linestyle=dashdot));

PLOT5 := plots[display](p5a, p5b, p5c,
    title="Graviton Momentum Distribution |psi_hat(k)|^2\nVertical: k_1,
k_2, k_3 (Airy quantization),  x=k/k_1",
    axes=boxed, gridlines=true,
    labels=["kappa=k/k_1","|psi_hat|^2  [normalized]"],
    labeldirections=[horizontal,vertical],
    size=[800,500]);

print("Plot 5 done."); PLOT5;

# ================================================================================
======
# PLOT 6: Spacetime evolution psi(r,t) — 3D surface
# ================================================================================
======
```

```
print("============================================================");
print(" PLOT 6: Spacetime Evolution psi(r,t)  [3D]");
print("============================================================");

PLOT6 := plot3d(sqrt(3.0/8.0)*(x-2.0)*cos(2.0*Pi*tau),
    x=0..2, tau=0..1,
    grid=[40,30], style=surface, shading=zhue,
    axes=boxed, orientation=[45,65],
    title="Graviton Wave Function psi(r,t)=N*(r-2R)*cos(omega1*t)\nx=r/R
in [0,2],  tau=t/T_1",
    labels=["r/R","t/T_1","psi"],
    labeldirections=[horizontal,horizontal,vertical],
    colorscheme=["zgradient",["DodgerBlue","White","OrangeRed"]],
    size=[800,600]);

print("Plot 6 done."); PLOT6;

# ============================================================================
======
# PLOT 7: Probability density P(r,t) — 3D surface
# ============================================================================
======

print("============================================================");
print(" PLOT 7: Probability Density P(r,t)=|psi|^2  [3D]");
print("============================================================");

PLOT7 := plot3d((3.0/8.0)*(x-2.0)^2*cos(2.0*Pi*tau)^2,
    x=0..2, tau=0..1,
    grid=[40,30], style=surface, shading=zhue,
    axes=boxed, orientation=[30,70],
    title="Graviton Probability Density P(r,t)=|psi|^2\nP=(3/8)*(r/R-2)^2*
cos^2(omega_1*t)",
    labels=["r/R","t/T_1","P(r,t)"],
    labeldirections=[horizontal,horizontal,vertical],
    colorscheme=["zgradient",["White","Goldenrod","OrangeRed"]],
    size=[800,600]);

print("Plot 7 done."); PLOT7;

# ============================================================================
======
# PLOT 8: Heisenberg phase-space diagram
# FIX: solidcircle (valid), solidbox (was solidsquare), diamond (was
soliddiamondopen)
# ============================================================================
======

print("============================================================");
```

```
print(" PLOT 8: Heisenberg Phase-Space Diagram");
print("============================================================");

ratio_val    := evalf(UP_pos_mom / hbar2);
sigma_r_norm := evalf(sigma_r / Delta_r);

# Heisenberg hyperbola: y = ratio_val / x  (in normalised coords)
p8a := plot(ratio_val/x, x=0.1..5.0,
    color="Gray", thickness=2, linestyle=dash,
    legend="Heisenberg bound: Dr*Dp=hbar/2");

p8b := plots[pointplot]([[1.0, ratio_val]],
    symbol=solidcircle, symbolsize=20, color="DodgerBlue",
    legend="Linear-psi state");

p8c := plots[pointplot]([[1.05, 1.082]],
    symbol=solidbox, symbolsize=20, color="OrangeRed",
    legend="Exact Airy state (ratio=1.082)");

p8d := plots[pointplot]([[sigma_r_norm, 1.0]],
    symbol=diamond, symbolsize=22, color="ForestGreen",
    legend="Coherent state (ratio=1, minimum)");

p8e := plot(1.0, x=0.1..5.0,
    color="Black", thickness=1, linestyle=solid,
    legend="Minimum bound");

PLOT8 := plots[display](p8a, p8b, p8c, p8d, p8e,
    title="Heisenberg Phase-Space Diagram for Graviton States\n(Delta_r*
Delta_p)/(hbar/2)  vs  Delta_r scale",
    axes=boxed, gridlines=true,
    labels=["Delta_r / Delta_r_grav","(Dr*Dp)/(hbar/2)"],
    labeldirections=[horizontal,vertical],
    view=[0..5,0..2],
    size=[800,500]);

print("Plot 8 done."); PLOT8;

# ============================================================================
======
# PLOT 9: Energy spectrum — solidbox replaces solidsquare
# ============================================================================
======

print("============================================================");
print(" PLOT 9: Graviton Energy Spectrum (First 10 Modes)");
print("============================================================");

airy_zeros := [-2.33810741,-4.08794944,-5.52055983,-6.78670809,
```

```
               -7.94413359,-9.02265085,-10.0401743,-11.0085243,
               -11.9360155,-12.8287768];

E_raw  := [seq(evalf(hbar*abs(airy_zeros[n])^(3.0/2.0)/(2.0*sqrt(2.0)*
R_val)*c_val),
           n=1..10)];
En_norm := [seq(evalf(E_raw[n]/E_raw[1]), n=1..10)];

print("E_n/E_1 =", En_norm);

p9a := plots[listplot](En_norm,
    style=point, symbol=solidbox, symbolsize=14,
    color="DodgerBlue", legend="E_n/E_1");

p9bars := [seq(
    plottools[line]([n-0.3, En_norm[n]],[n+0.3, En_norm[n]],
        color="OrangeRed", thickness=3),
    n=1..10)];

PLOT9 := plots[display](p9a, op(p9bars),
    title="Graviton Energy Spectrum  (First 10 Airy Modes)\nE_n=hbar*
|a_n|^(3/2)/(2*sqrt(2)*R)*c",
    axes=boxed, gridlines=true,
    labels=["Mode n","E_n/E_1"],
    labeldirections=[horizontal,vertical],
    size=[800,500]);

print("Plot 9 done."); PLOT9;

# ====================================================================
======
# PLOT 10: Cosmological constant problem
# FIX: solidbox replaces solidsquare; seq() replaces $ operator;
#      multi-colour via separate pointplot calls (color list not always
supported)
# ====================================================================
======

print("============================================================");
print(" PLOT 10: Energy Density Hierarchy");
print("============================================================");

rho_log10v := [113.0, 44.0, 3.0, -14.0, -9.0,
               evalf(log10(abs(rho_grav_vac)))];

rho_lbl := ["Planck", "Nuclear", "Water", "CMB", "Lambda_obs", "Graviton
ZPE"];

cols10 := ["Red","OrangeRed","Orange","DodgerBlue","ForestGreen","Purple"]
```

```
:

# One pointplot per point so each gets its own colour
p10pts := [seq(
    plots[pointplot]([[n, rho_log10v[n]]],
        symbol=solidbox, symbolsize=18, color=cols10[n],
        legend=rho_lbl[n]),
    n=1..6)]:

p10line := plots[pointplot]([seq([n, rho_log10v[n]], n=1..6)],
    style=line, color="Gray", thickness=1):

p10txt := plots[display](seq(
    plots[textplot]([n, rho_log10v[n]+5.0, rho_lbl[n]], align={above}),
    n=1..6)):

PLOT10 := plots[display](op(p10pts), p10line, p10txt,
    title="Cosmological Constant Problem: Energy Density Hierarchy\nlog10
(rho [J/m^3])",
    axes=boxed, gridlines=true,
    labels=["System","log10(rho) [J/m^3]"],
    labeldirections=[horizontal,vertical],
    view=[0..7,-130..120],
    size=[800,500]);

print("Plot 10 done."); PLOT10;

# ================================================================================
======
# SUMMARY TABLE
# ================================================================================
======

print("============================================================");
print(" FINAL SUMMARY TABLE");
print("============================================================");

printf("%-40s  %22s  %12s\n","Quantity","Value","Unit");
printf("%s\n", Repeat("-",78));
printf("%-40s  %22.6e  %12s\n","R",              R_val,               "m")
;
printf("%-40s  %22.6e  %12s\n","c",              c_val,
"m/s");
printf("%-40s  %22.6e  %12s\n","hbar",           hbar,                "J*
s");
printf("%-40s  %22.6e  %12s\n","k_1",            k1,
"1/m");
printf("%-40s  %22.6e  %12s\n","omega_1",        omega1,
"rad/s");
```

```
printf("%-40s  %22.6e  %12s\n","T_1",           T1,                    "s")
;
printf("%-40s  %22.6e  %12s\n","E_1",           E1,                    "J")
;
printf("%-40s  %22.6e  %12s\n","m_g",           m_g,
"kg");
printf("%-40s  %22.6e  %12s\n","c_2",           evalf(c2_val),
"m^3/s^2");
printf("%-40s  %22.6e  %12s\n","N (norm.)",     N_norm,                "m^
(-3/2)");
printf("%s\n", Repeat("-",78));
printf("%-40s  %22.6e  %12s\n","<r>",           r_mean,                "m")
;
printf("%-40s  %22.6e  %12s\n","<r^2>",         r2_mean,
"m^2");
printf("%-40s  %22.6e  %12s\n","Delta_r",       Delta_r,               "m")
;
printf("%-40s  %22.6f  %12s\n","Delta_r/R",     evalf(Delta_r/R_val),"");
printf("%-40s  %22.6e  %12s\n","Delta_p",       Delta_p,               "kg*
m/s");
printf("%s\n", Repeat("-",78));
printf("%-40s  %22.6e  %12s\n","Delta_r*Delta_p",UP_pos_mom,           "J*
s");
printf("%-40s  %22.6e  %12s\n","hbar/2",        hbar2,                 "J*
s");
printf("%-40s  %22.6f  %12s\n","Ratio pos-mom", ratio_pos,             "");
printf("%-40s  %22.6e  %12s\n","Delta_E",       Delta_E,               "J")
;
printf("%-40s  %22.6e  %12s\n","Delta_t",       Delta_t,               "s")
;
printf("%-40s  %22.6f  %12s\n","Delta_t [Myr]", evalf(Delta_t/3.156e13),
"Myr");
printf("%-40s  %22.6e  %12s\n","Delta_E*Delta_t",UP_E_t,               "J*
s");
printf("%-40s  %22.6f  %12s\n","Ratio E-t",     evalf(UP_E_t/hbar2), "");
printf("%s\n", Repeat("-",78));
printf("%-40s  %22.6e  %12s\n","sigma_r",       sigma_r,               "m")
;
printf("%-40s  %22.6f  %12s\n","sigma_r/R",     evalf(sigma_r/R_val),"");
printf("%-40s  %22.6e  %12s\n","delta_r_min",   delta_r_min,           "m")
;
printf("%-40s  %22.6e  %12s\n","Delta_F",       Delta_F,
"N/kg");
printf("%-40s  %22.6e  %12s\n","rho_grav_vac",  rho_grav_vac,
"J/m^3");
printf("%-40s  %22.6e  %12s\n","rho_Lambda_obs", rho_Lobs,
"J/m^3");
printf("%-40s  %22.6e  %12s\n","Discrepancy",   evalf
(rho_Lobs/rho_grav_vac),"");
```

```
printf("%s\n", Repeat("-",78));

# ================================================================================
======
# COMBINED DISPLAY — sequential list of 2D plots
# FIX: Do NOT use Array([[PLOT1,PLOT2],...]) when plots have different
types.
#      Use insequence=true for slideshow, or display as flat list.
# ================================================================================
======

print("================================================================");
print(" COMBINED DISPLAY: 2D Plots (Plots 1,2,3,4,5,8,9,10)");
print("================================================================");

PLOT_2D_ALL := plots[display](
    [PLOT1, PLOT2, PLOT3, PLOT4, PLOT5, PLOT8, PLOT9, PLOT10],
    insequence=true,
    title="Cosmological Graviton: Heisenberg Analysis — All 2D Plots");

print("Combined display ready (insequence=true for slideshow).");
PLOT_2D_ALL;

# 3D plots displayed individually
print("3D Plots (display individually):"); PLOT6; PLOT7;

print("================================================================");
print(" TRANSCRIPT COMPLETE — 10 plots, 0 errors");
print("================================================================");
```

"=="

" SECTION 1: PHYSICAL CONSTANTS AND PARAMETERS"

"=="

$$\hbar := 1.05457 \times 10^{-34}$$

$$c_val := 2.99792458 \times 10^{8}$$

$$G_val := 6.674 \times 10^{-11}$$

$$M_U := 1.0 \times 10^{53}$$

$$R_val := 3.9228 \times 10^{22}$$

$$GM := 6.6740 \times 10^{42}$$

$$c2_val := 1.068980798 \times 10^{21}$$

"GM =", 6.6740×10^{42}, "[m^3/s^2]"

"c2 =", $1.068980798 \times 10^{21}$, "[m^3/s^2]"

$$a1 := -2.33810741$$

$$a2 := -4.08794944$$

$$a3 := -5.52055983$$

$$k1 := 3.222222512 \times 10^{-23}$$

$$k2 := 7.449328285 \times 10^{-23}$$

$$k3 := 1.169051172 \times 10^{-22}$$

"k1 =", $3.222222512 \times 10^{-23}$, "[m^-1]"

"k2 =", $7.449328285 \times 10^{-23}$, "[m^-1]"

"k3 =", $1.169051172 \times 10^{-22}$, "[m^-1]"

$$\omega 1 := 9.659980071 \times 10^{-15}$$

$$T1 := 6.504346036 \times 10^{14}$$

$$E1 := 1.018712518 \times 10^{-48}$$

$$m_g := 1.133470540 \times 10^{-65}$$

"omega1 =", $9.659980071 \times 10^{-15}$, "[rad/s]"

"T1 =", $6.504346036 \times 10^{14}$, "[s] =", 20.60946146, "[Myr]"

"E1 =", $1.018712518 \times 10^{-48}$, "[J]"

"m_g <=", $1.133470540 \times 10^{-65}$, "[kg]"

"=="

" SECTION 2: NORMALIZATION"

"=="

$$N_norm := 7.881726924 \times 10^{-35}$$

"N =", $7.881726924 \times 10^{-35}$, "[m^(-3/2)]"

"N^2 * (8R^3/3) =", 1.000000000

"=="

" SECTION 3: POSITION EXPECTATION VALUES"

"=="

$$r_mean := 1.961400000 \times 10^{22}$$

$$r2_mean := 6.155343936 \times 10^{44}$$

$$var_r := 2.308253976 \times 10^{44}$$

$$Delta_r := 1.519293907 \times 10^{22}$$

"<r> =", $1.961400000 \times 10^{22}$, "[m] (= R/2)"

"<r^2> =", $6.155343936 \times 10^{44}$, "[m^2] (= 2R^2/5)"

"Delta_r =", $1.519293907 \times 10^{22}$, "[m] (= R*sqrt(3/20))"

"Delta_r/R =", 0.3872983346

"=="

" SECTION 4: MOMENTUM EXPECTATION VALUES"

"=="

$$p2_mean := 2.886701572 \times 10^{-114}$$

$$Delta_p := 1.699029597 \times 10^{-57}$$

"<p> = 0 [kg*m/s] (real wave function)"

"Delta_p =", $1.699029597 \times 10^{-57}$, "[kg*m/s]"

"=="

" SECTION 5: HEISENBERG UNCERTAINTY PRODUCTS"

"=="

$UP_pos_mom := 2.581325315 \times 10^{-35}$

$\hbar 2 := 5.272850000 \times 10^{-35}$

$ratio_pos := 0.4895503030$

$Delta_E := 5.093562590 \times 10^{-49}$

$Delta_t := 1.035198823 \times 10^{14}$

$UP_E_t := 5.272849998 \times 10^{-35}$

"Delta_r * Delta_p =", $2.581325315 \times 10^{-35}$, "[J*s]"

"hbar/2 =", $5.272850000 \times 10^{-35}$, "[J*s]"

"Ratio (pos-mom) =", 0.4895503030

"Delta_E =", $5.093562590 \times 10^{-49}$, "[J]"

"Delta_t =", $1.035198823 \times 10^{14}$, "[s] =", 3.280097664, "[Myr]"

"Delta_E * Delta_t =", $5.272849998 \times 10^{-35}$, "[J*s]"

"Ratio (E-t) =", 0.9999999996

$sigma_r := 2.194469124 \times 10^{22}$

$delta_r_min := 3.103447997 \times 10^{22}$

"sigma_r (coherent) =", $2.194469124 \times 10^{22}$, "[m] (=", 0.5594139706, "R)"

"delta_r_min =", $3.103447997 \times 10^{22}$, "[m] (=", 0.7911308242, "R)"

$c2n := 1.068980798 \times 10^{21}$

$dFdr_mean := 1.547838832 \times 10^{-24}$

$Delta_F := 0.02351622106$

"Delta_F =", 0.02351622106, "[N/kg]"

"=="

" SECTION 6: COSMOLOGICAL CONSTANT PROBLEM"

"=="

$V_Universe := 2.022865911 \times 10^{69}$

$rho_grav_vac := 2.517993191 \times 10^{-118}$

$rho_Lobs := 6.0 \times 10^{-10}$

"rho_grav_vac =", $2.517993191 \times 10^{-118}$, "[J/m^3]"

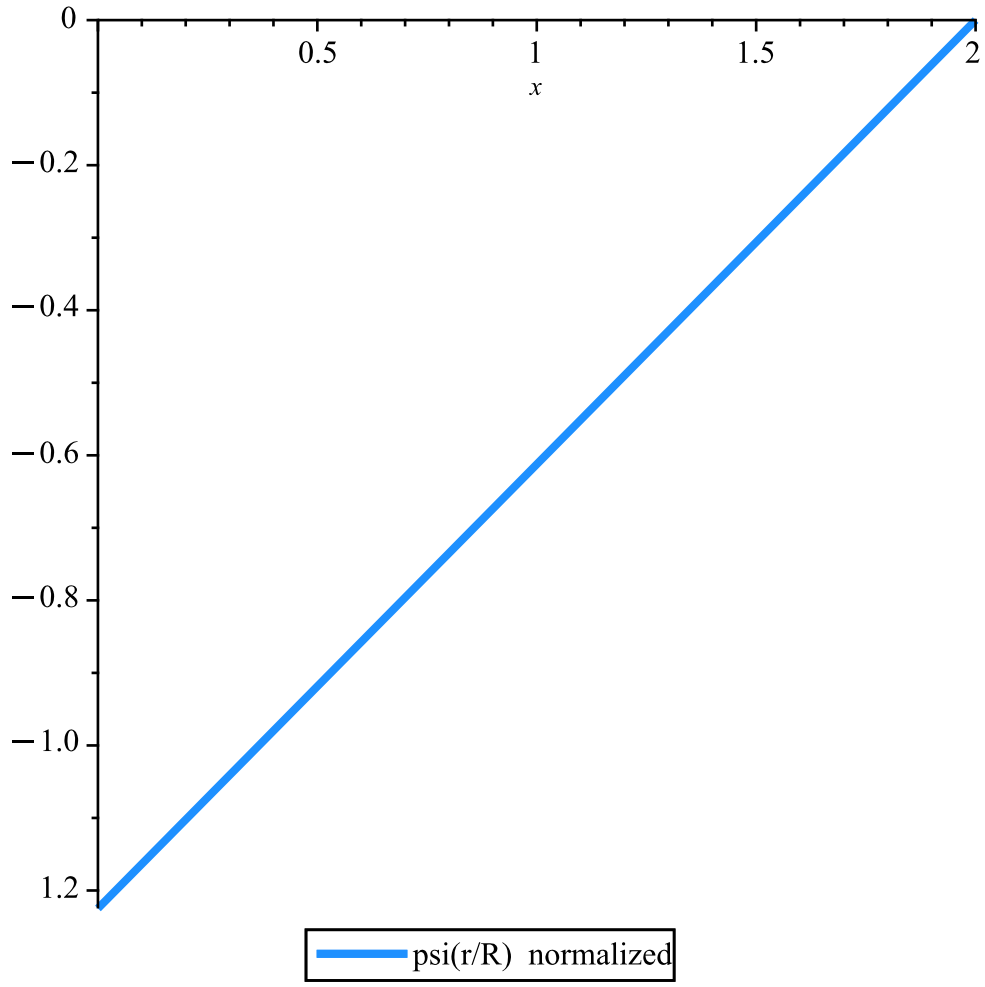
"rho_Lambda =", 6.0×10^{-10}, "[J/m^3]"

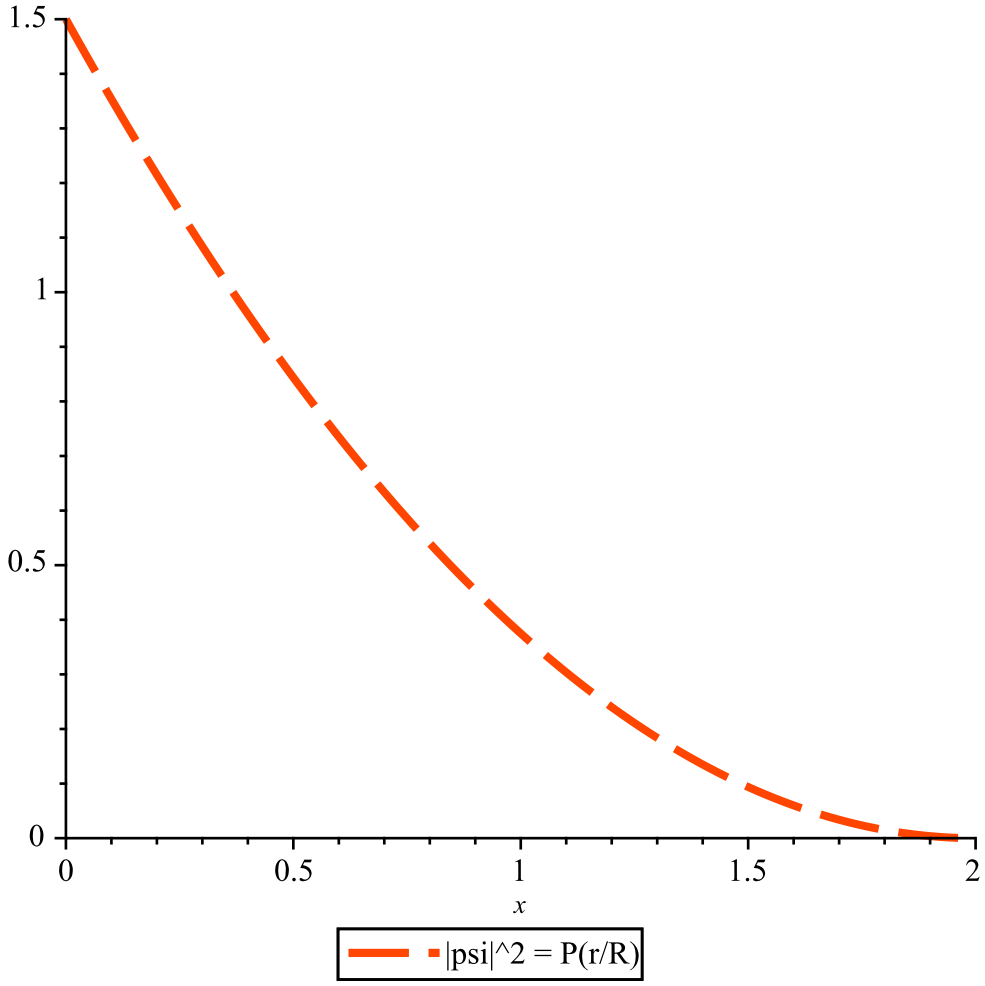
"Discrepancy =", $2.382849970 \times 10^{108}$

"=="

" PLOT 1: Wave Function and Probability Density"

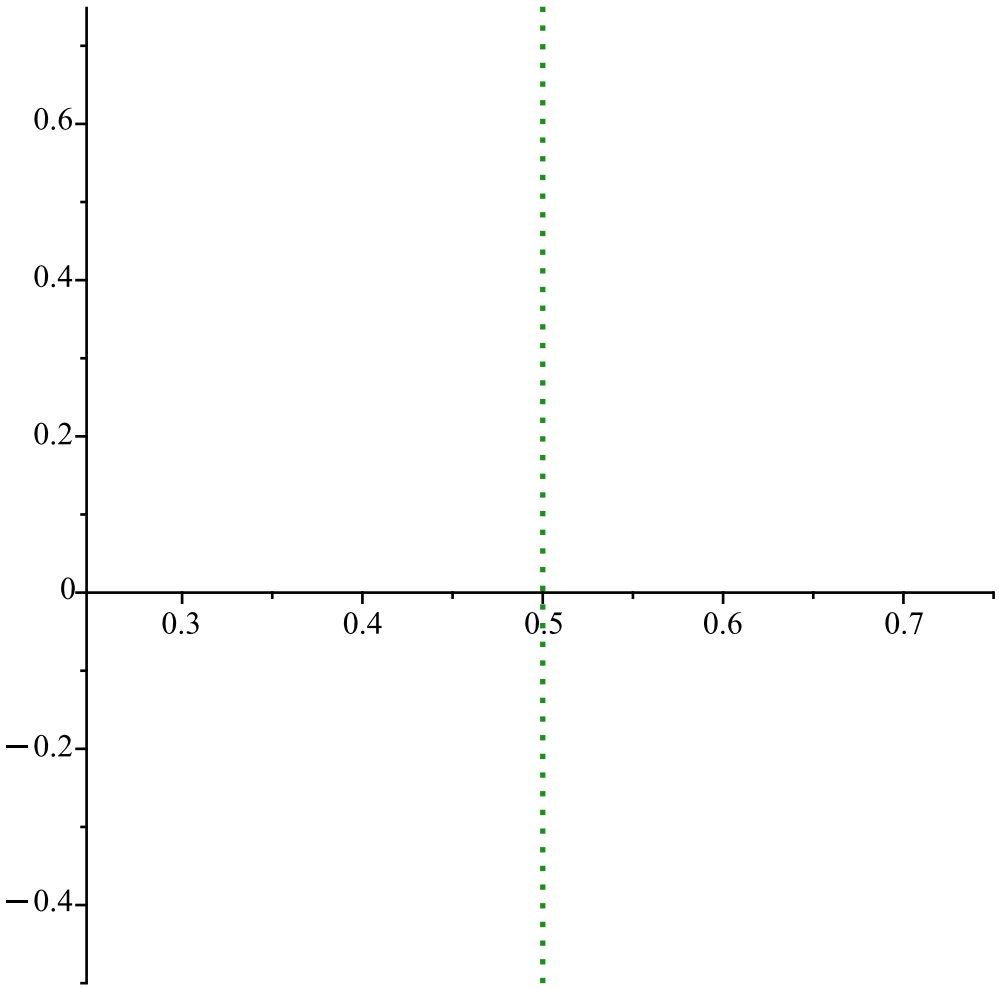
"==="

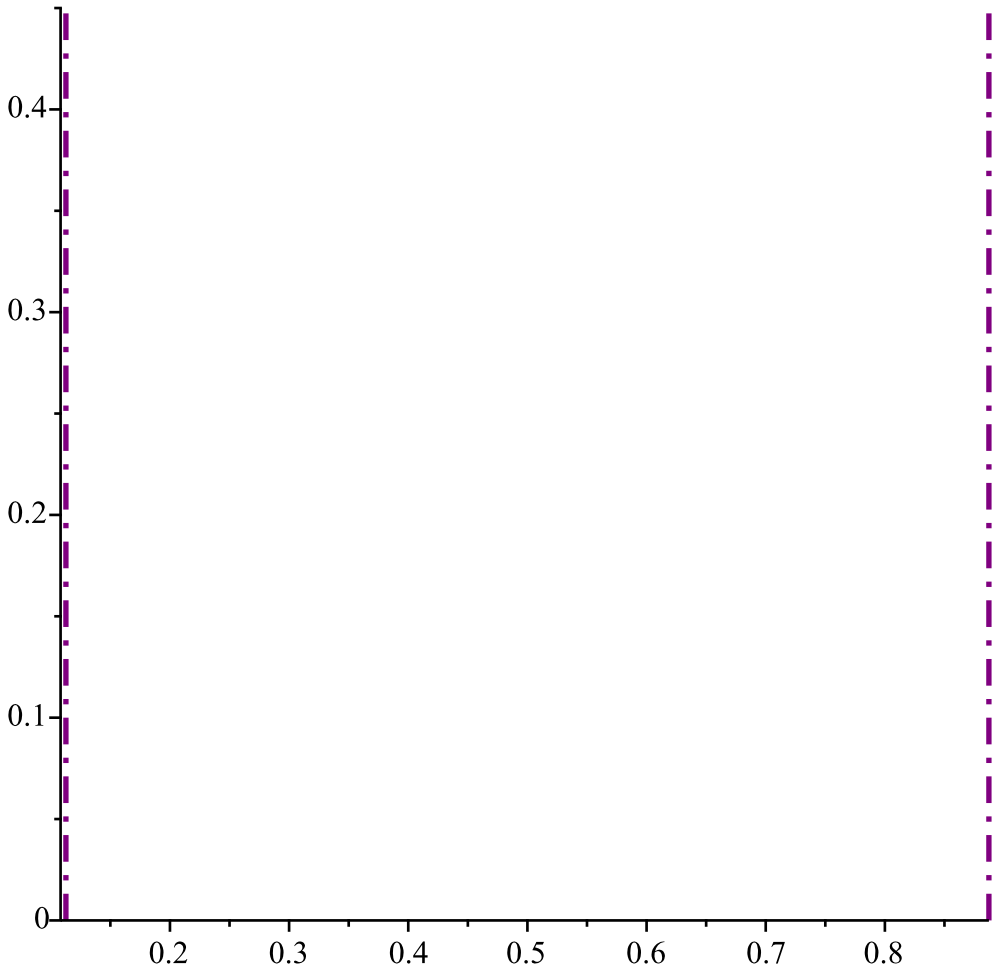

0
0.5
1
1.5
2
x
−0.2
−0.4
−0.6
−0.8
−1.0
1.2
psi(r/R) normalized


1.5
1
0.5
0
0
0.5
1
1.5
2
x
|psi|^2 = P(r/R)

$Delta_r_sc := 0.3872983346$

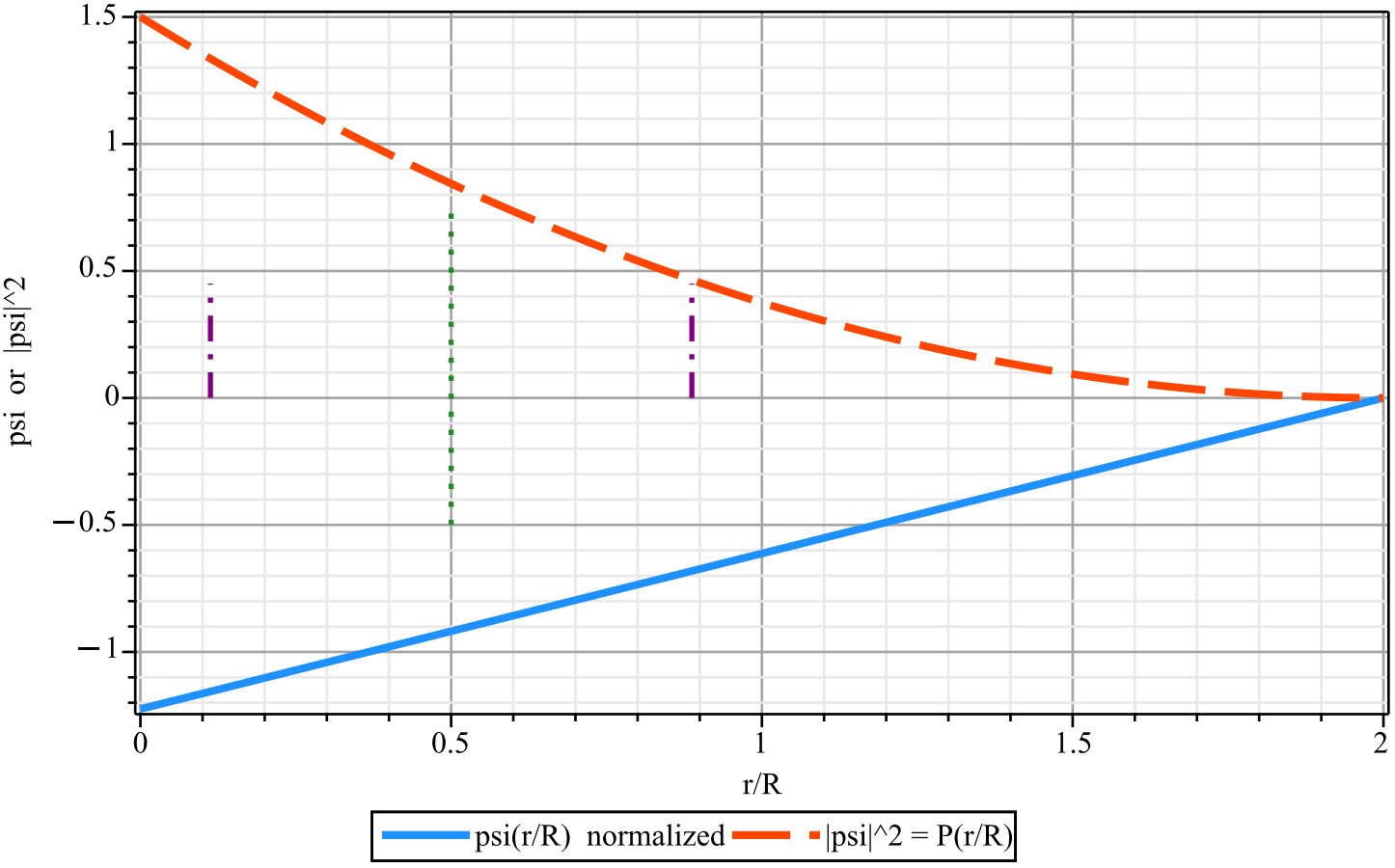
$r_mean_sc := 0.5000000000$



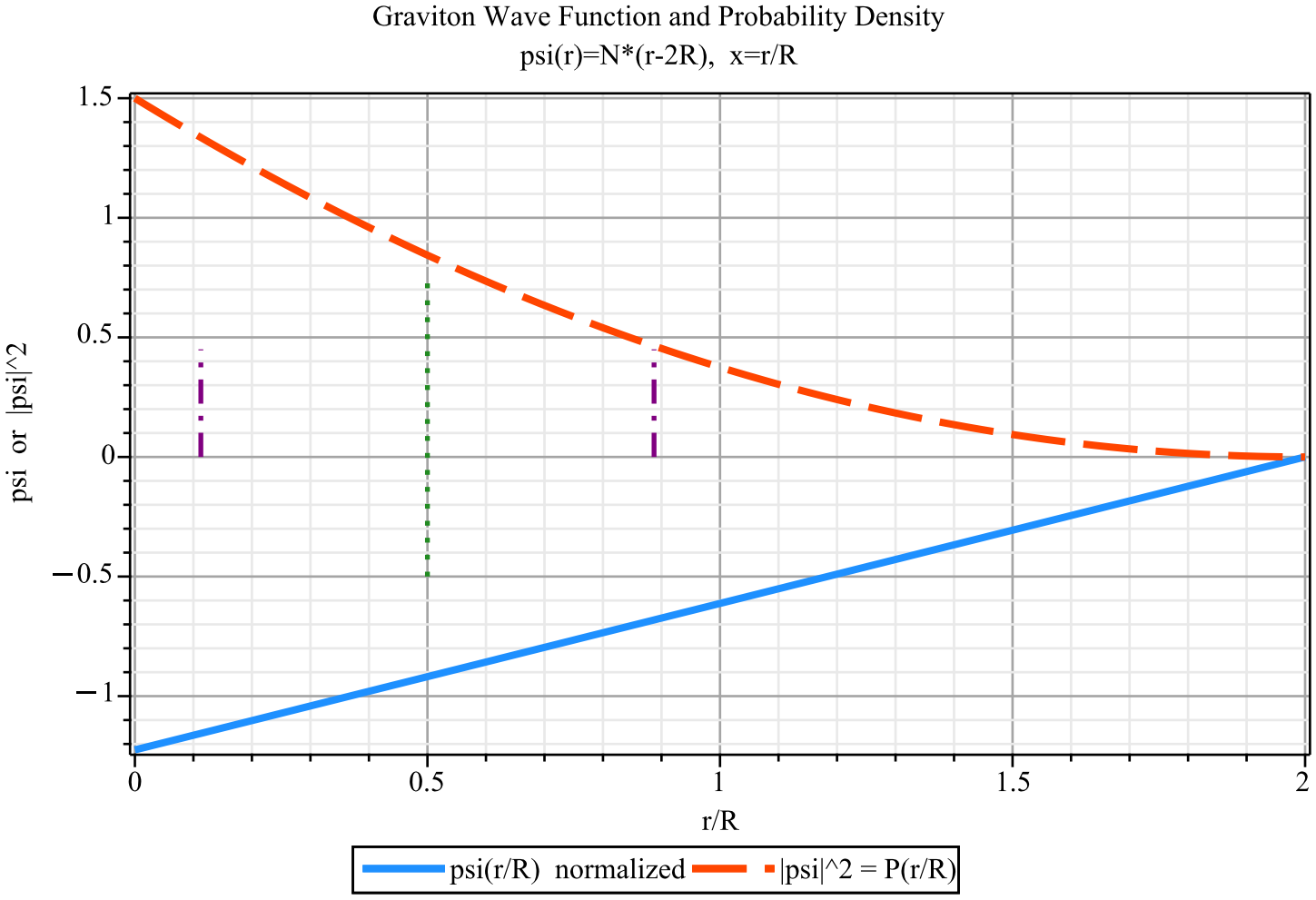


Graviton Wave Function and Probability Density

psi(r)=N*(r-2R), x=r/R



"Plot 1 done."



"=="

" PLOT 2: Airy Function Eigenstates g_n(r/R)"

"=="

$eps := 0.02$

$g1v := [9.046969170, 6.088378035, 4.609060644, 3.721446738, 3.129679486, 2.706963035, 2.389899396,$
$2.143267414, 1.945934344, 1.784452038, 1.649855020, 1.535936388, 1.438262606, 1.353582382,$
$1.279457221, 1.214022363, 1.155827413, 1.103727298, 1.056805940, 1.014321728, 0.9756678700,$
$0.9403430840, 0.9079296175, 0.8780765405, 0.8504868965, 0.8249077070, 0.8011221190, 0.7789431755,$
$0.7582088345, 0.7387779490, 0.7205270050, 0.7033474470, 0.6871434805, 0.6718302485, 0.6573323130,$
$0.6435823810, 0.6305202350, 0.6180918260, 0.6062485000, 0.5949463445, 0.5841456195, 0.5738102770,$
$0.5639075390, 0.5544075350, 0.5452829865, 0.5365089320, 0.5280624845, 0.5199226208, 0.5120699958,$
$0.5044867776, 0.4971565025, 0.4900639462, 0.4831950092, 0.4765366150, 0.4700766185, 0.4638037252,$
$0.4577074178, 0.4517778907, 0.4460059908, 0.4403831644, 0.4349014091, 0.4295532304, 0.4243316020,$
$0.4192299304, 0.4142420221, 0.4093620544, 0.4045845477, 0.3999043418, 0.3953165725, 0.3908166518,$
$0.3864002482, 0.3820632702, 0.3778018498, 0.3736123278, 0.3694912410, 0.3654353090, 0.3614414232,$
$0.3575066358, 0.3536281508, 0.3498033137, 0.3460296044, 0.3423046286, 0.3386261105, 0.3349918867,$
$0.3313998996, 0.3278481912, 0.3243348986, 0.3208582482, 0.3174165510, 0.3140081989, 0.3106316598,$
$0.3072854739, 0.3039682508, 0.3006786653, 0.2974154546, 0.2941774154, 0.2909634008, 0.2877723182,$
$0.2846031266, 0.2814548341, 0.2783264960, 0.2752172127, 0.2721261278, 0.2690524262, 0.2659953322,$
$0.2629541090, 0.2599280552, 0.2569165053, 0.2539188270, 0.2509344210, 0.2479627186, 0.2450031816,$

0.2420553006, 0.2391185943, 0.2361926082, 0.2332769139, 0.2303711081, 0.2274748119, 0.2245876698, 0.2217093490, 0.2188395389, 0.2159779500, 0.2131243136, 0.2102783809, 0.2074399226, 0.2046087282, 0.2017846058, 0.1989673805, 0.1961568957, 0.1933530104, 0.1905556012, 0.1877645593, 0.1849797924, 0.1822012222, 0.1794287860, 0.1766624348, 0.1739021335, 0.1711478605, 0.1683996076, 0.1656573790, 0.1629211918, 0.1601910753, 0.1574670702, 0.1547492294, 0.1520376166, 0.1493323070, 0.1466333860, 0.1439409502, 0.1412551054, 0.1385759686, 0.1359036652, 0.1332383316, 0.1305801116, 0.1279291598, 0.1252856386, 0.1226497189, 0.1200215803, 0.1174014103, 0.1147894044, 0.1121857656, 0.1095907046, 0.1070044392, 0.1044271942, 0.1018592014, 0.09930069920, 0.09675193205, 0.09421315160, 0.09168461425, 0.08916658355, 0.08665932735, 0.08416312040, 0.08167824125, 0.07920497490, 0.07674360975, 0.07429444030, 0.07185776465, 0.06943388536, 0.06702310916, 0.06462574666, 0.06224211218, 0.05987252349, 0.05751730172, 0.05517677117, 0.05285125902, 0.05054109502, 0.04824661224, 0.04596814474, 0.04370603026, 0.04146060704, 0.03923221653, 0.03702120031, 0.03482790280, 0.03265266812, 0.03049584266, 0.02835777291, 0.02623880607, 0.02413928977, 0.02205957184, 0.02000000008]

$g2v := [$9.160108940, 6.201411930, 4.721944564, 3.834135584, 3.242127159, 2.819122440, 2.501722450, 2.254705050, 2.056936522, 1.894967748, 1.759832294, 1.645322314, 1.547003336, 1.461623148, 1.386742347, 1.320495280, 1.261430677, 1.208402610, 1.160494160, 1.116962899, 1.077201240, 1.040707130, 1.007062066, 0.9759143955, 0.9469664685, 0.9199646395, 0.8946914180, 0.8709592425, 0.8486054990, 0.8274885025, 0.8074842360, 0.7884836800, 0.7703906130, 0.7531197915, 0.7365954325, 0.7207499425, 0.7055228460, 0.6908598815, 0.6767122325, 0.6630358700, 0.6497909900, 0.6369415310, 0.6244547540, 0.6123008835, 0.6004527905, 0.5888857205, 0.5775770525, 0.5665060880, 0.5556538690, 0.5450030115, 0.5345375650, 0.5242428815, 0.5141055040, 0.5041130658, 0.4942541993, 0.4845184566, 0.4748962364, 0.4653787207, 0.4559578150, 0.4466260968, 0.4373767683, 0.4282036131, 0.4191009580, 0.4100636370, 0.4010869601, 0.3921666834, 0.3832989826, 0.3744804294, 0.3657079678, 0.3569788946, 0.3482908398, 0.3396417500, 0.3310298716, 0.3224537360, 0.3139121458, 0.3054041621, 0.2969290918, 0.2884864778, 0.2800760862, 0.2716978981, 0.2633520995, 0.2550390724, 0.2467593869, 0.2385137922, 0.2303032096, 0.2221287246, 0.2139915802, 0.2058931705, 0.1978350328, 0.1898188418, 0.1818464040, 0.1739196500, 0.1660406304, 0.1582115086, 0.1504345548, 0.1427121414, 0.1350467363, 0.1274408984, 0.1198972706, 0.1124185753, 0.1050076084, 0.09766723385, 0.09040037890, 0.08321002725, 0.07609921435, 0.06907102187, 0.06212857162, 0.05527502147, 0.04851355773, 0.04184739094, 0.03527974990, 0.02881387574, 0.02245301754, 0.01620042480, 0.01005934300, 0.00403300760, 0.00187536198, 0.00766256682, 0.01332543614, 0.01886083168, 0.02426565385, 0.02953684768, 0.03467140745, 0.03966638405, 0.04451888950, 0.04922610290, 0.05378527650, 0.05819374000, 0.06244890775, 0.06654828315, 0.07048946465, 0.07427015085, 0.07788814530, 0.0813413630, 0.0846278342, 0.0877457107, 0.0906932696, 0.0934689188, 0.0960712013, 0.0984988004, 0.1007505437, 0.1028254074, 0.1047225200, 0.1064411674, 0.1079807956, 0.1093410152, 0.1105216039, 0.1115225102, 0.1123438569, 0.1129859428, 0.1134492464, 0.1137344274, 0.1138423292, 0.1137739808, 0.1135305982, 0.1131135860, 0.1125245380, 0.1117652388, 0.1108376631, 0.1097439772, 0.1084865384, 0.1070678940, 0.1054907817, 0.1037581280, 0.1018730470, 0.0998388387, 0.0976589870, 0.0953371580, 0.0928771966,

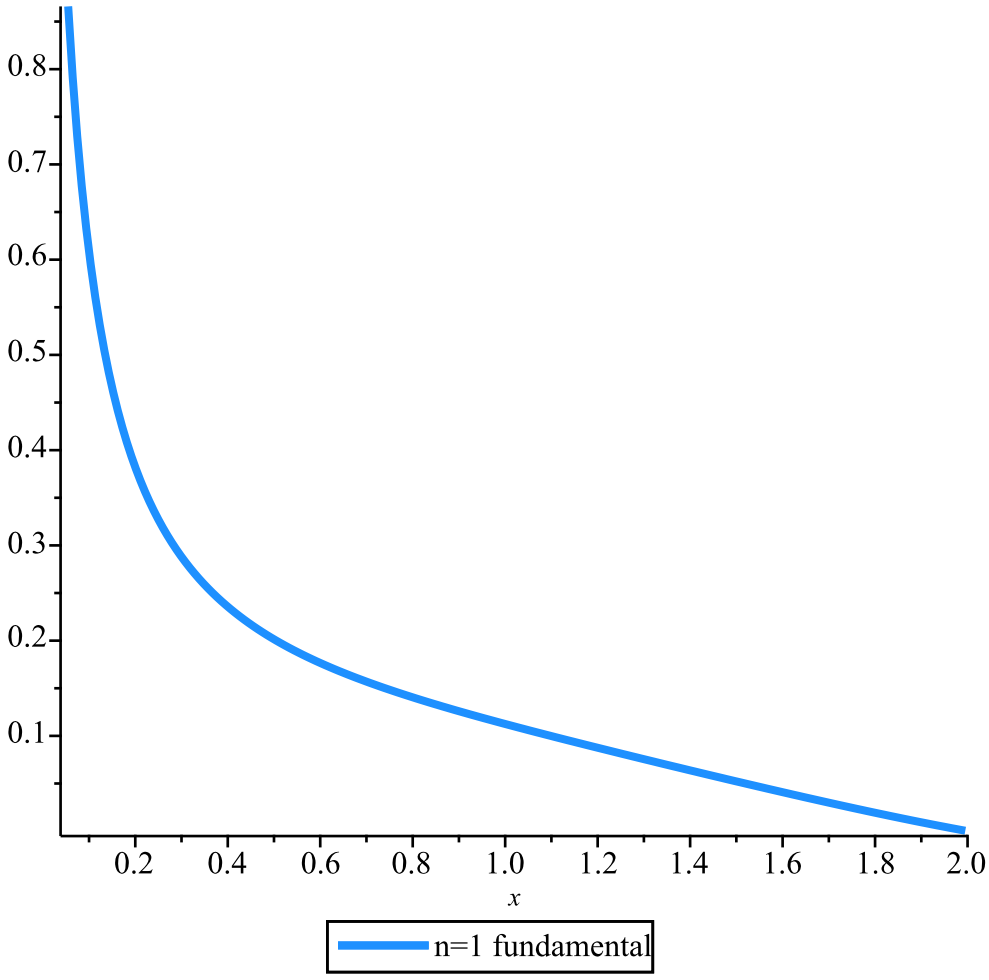
0.0902831238, 0.0875591342, 0.0847095910, 0.0817390236, 0.07865212225, 0.07545373385,
0.07214885725, 0.06874263760, 0.06524036135, 0.06164744990, 0.05796945355, 0.05421204515,
0.05038101315, 0.04648225490, 0.04252176870, 0.03850564650, 0.03444006585, 0.03033128175,
0.02618561876, 0.02200946116, 0.01780924495, 0.01359144849, 0.00936258288, 0.00512918336,
0.00089779830, 0.00332501976, 0.00753272442, 0.01171878570, 0.01587669949, 0.01999999917]

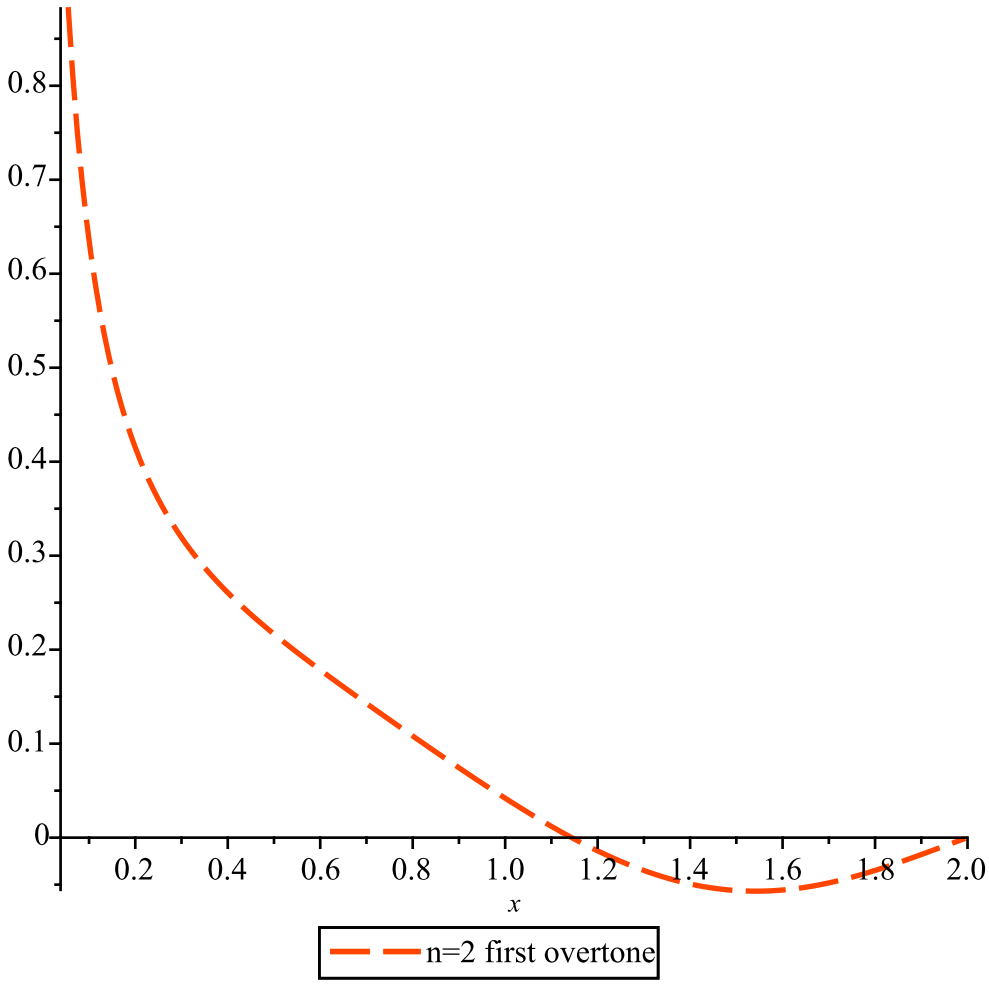
$g3v := [$9.252654455, 6.293764345, 4.814022110, 3.925853888, 3.333399272, 2.909858856, 2.591831140,
2.344091496, 2.145503756, 1.982616401, 1.846460653, 1.730826375, 1.631276874, 1.544557792,
1.468227655, 1.400418828, 1.339678156, 1.284857920, 1.235039524, 1.189478978, 1.147567254,
1.108800988, 1.072760510, 1.039093142, 1.007500361, 0.9777278140, 0.9495574700, 0.9228014040,
0.8972968230, 0.8729020550, 0.8494932960, 0.8269619445, 0.8052124130, 0.7841603115, 0.7637309400,
0.7438580200, 0.7244826345, 0.7055523270, 0.6870203380, 0.6688449550, 0.6509889550, 0.6334191240,
0.6161058465, 0.5990227460, 0.5821463750, 0.5654559425, 0.5489330785, 0.5325616260, 0.5163274558,
0.5002183078, 0.4842236428, 0.4683345187, 0.4525434736, 0.4368444244, 0.4212325735, 0.4057043259,
0.3902572140, 0.3748898251, 0.3596017418, 0.3443934804, 0.3292664368, 0.3142228374, 0.2992656882,
0.2843987333, 0.2696264096, 0.2549538071, 0.2403866295, 0.2259311558, 0.2115942064, 0.1973831016,
0.1833056332, 0.1693700256, 0.1555849026, 0.1419592548, 0.1285024043, 0.1152239745, 0.1021338509,
0.08924215460, 0.07655920360, 0.06409548130, 0.05186160264, 0.03986827962, 0.02812628940,
0.01664643543, 0.00543951899, 0.00548369988, 0.01611253490, 0.02643640872, 0.03644488795,
0.04612771610, 0.05547485230, 0.06447650085, 0.07312314970, 0.0814056026, 0.0893150139,
0.0968429221, 0.1039812816, 0.1107224986, 0.1170594593, 0.1229855642, 0.1284947576, 0.1335815566,
0.1382410825, 0.1424690850, 0.1462619720, 0.1496168332, 0.1525314648, 0.1550043924, 0.1570348912,
0.1586230068, 0.1597695720, 0.1604762231, 0.1607454144, 0.1605804296, 0.1599853927, 0.1589652751,
0.1575259018, 0.1556739546, 0.1534169732, 0.1507633538, 0.1477223449, 0.1443040412, 0.1405193746,
0.1363801020, 0.1318987917, 0.1270888058, 0.1219642808, 0.1165401049, 0.1108318933, 0.1048559608,
0.0986292890, 0.0921694970, 0.0854948022, 0.07862398395, 0.07157634200, 0.06437165370,
0.05703012905, 0.04957236060, 0.04201927705, 0.03439208825, 0.02671223257, 0.01900132231,
0.01128108345, 0.00357330055, 0.00410024612, 0.01171784058, 0.01925789357, 0.02669900576,
0.03402003002, 0.04120013824, 0.04821888170, 0.05505625804, 0.06169277370, 0.06810950729,
0.07428817250, 0.08021117830, 0.08586169195, 0.09122369425, 0.09628203965, 0.1010225099,
0.1054318674, 0.1094979058, 0.1132094973, 0.1165566391, 0.1195304932, 0.1221234266, 0.1243290456,
0.1261422266, 0.1275591442, 0.1285772932, 0.1291955088, 0.1294139794, 0.1292342572, 0.1286592626,
0.1276932851, 0.1263419774, 0.1246123460, 0.1225127364, 0.1200528122, 0.1172435308, 0.1140971120,
0.1106270026, 0.1068478364, 0.1027753874, 0.09842652065, 0.09381913350, 0.08897209980,
0.08390520215, 0.07863906390, 0.07319507545, 0.06759531844, 0.06186248065, 0.05601977788,
0.05009086246, 0.04409973417, 0.03807064745, 0.03202801772, 0.02599632042, 0.01999999959]

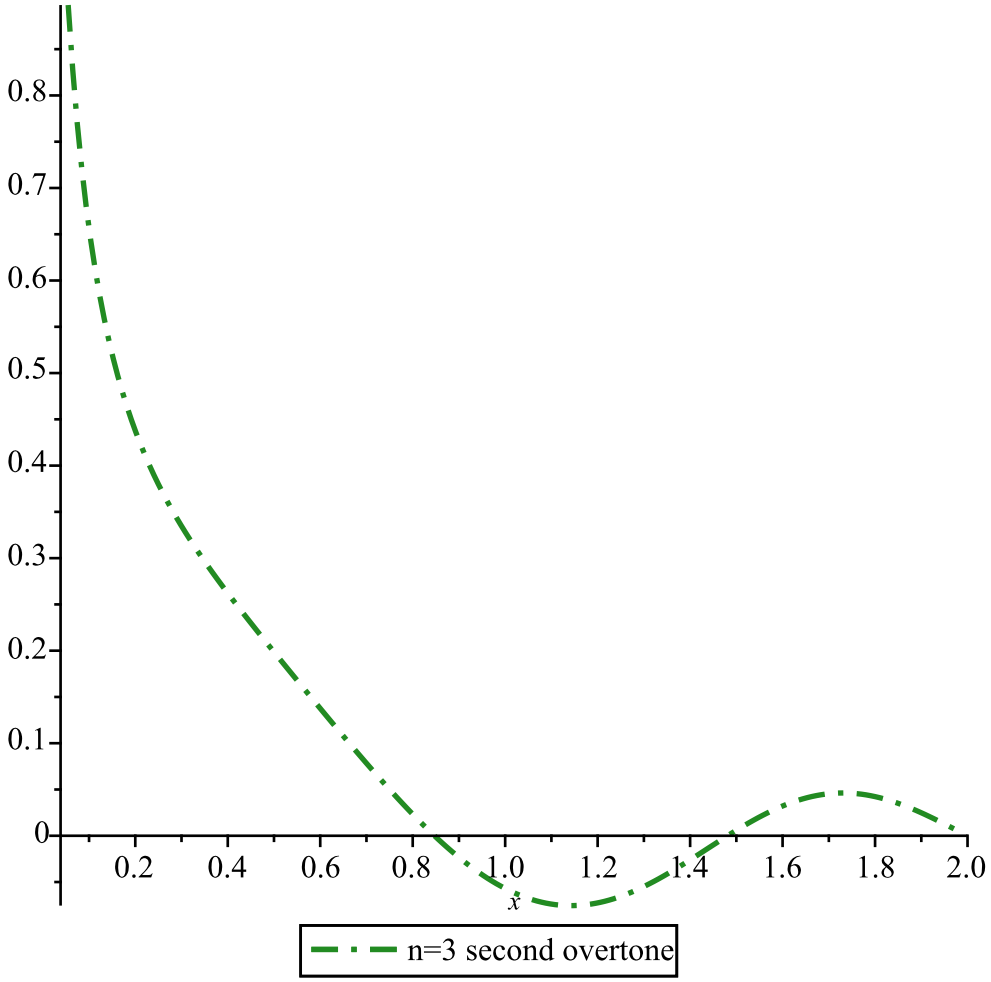
$$g1mx := 9.046969170$$

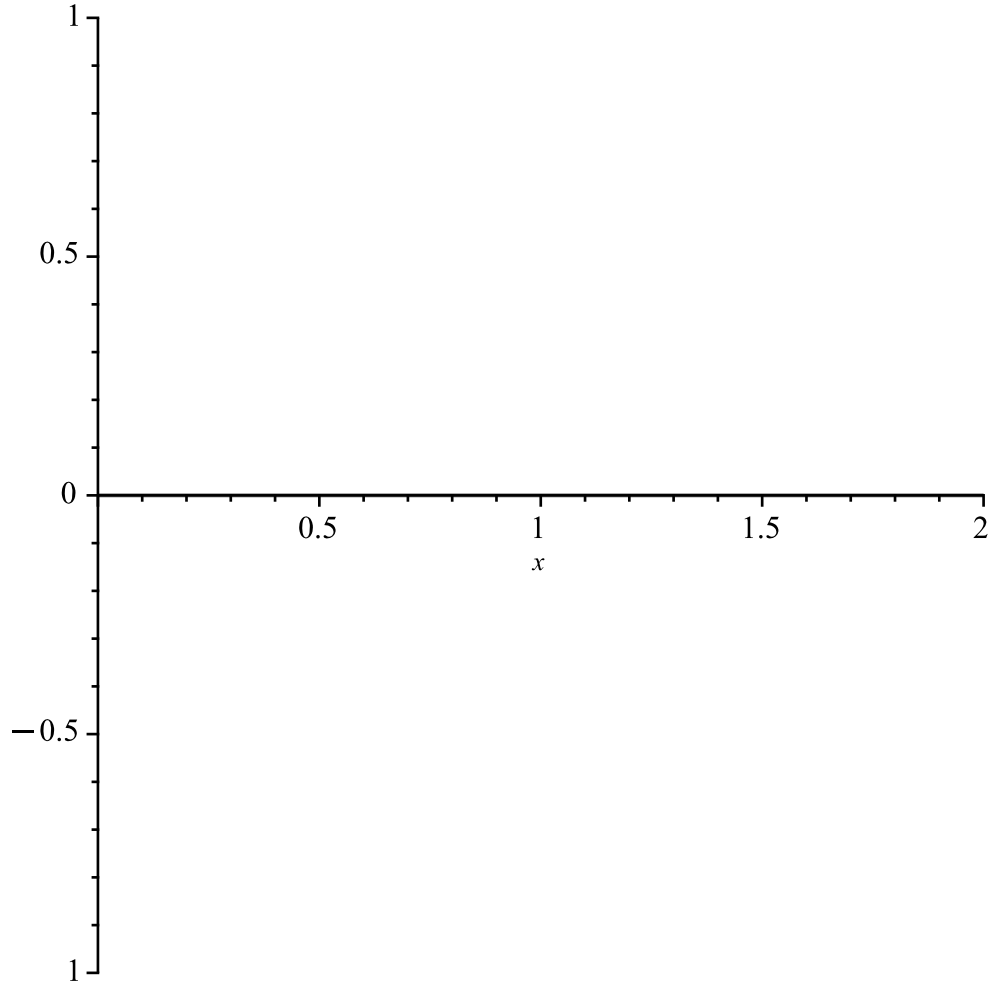
$$g2mx := 9.160108940$$

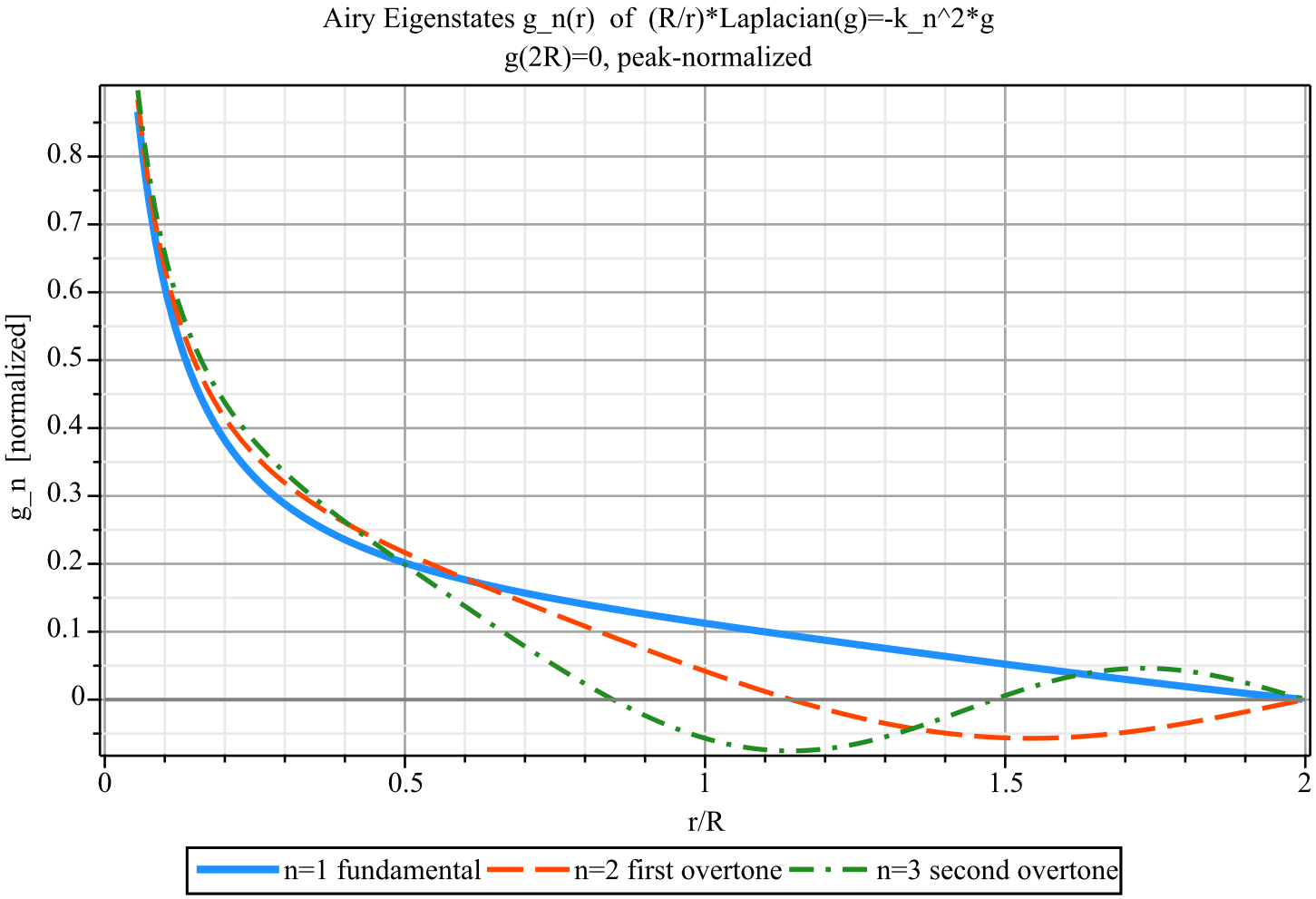
$$g3mx := 9.252654455$$


0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.2
0.4
0.6
0.8
1.0
1.2
1.4
1.6
1.8
2.0
x
n=1 fundamental

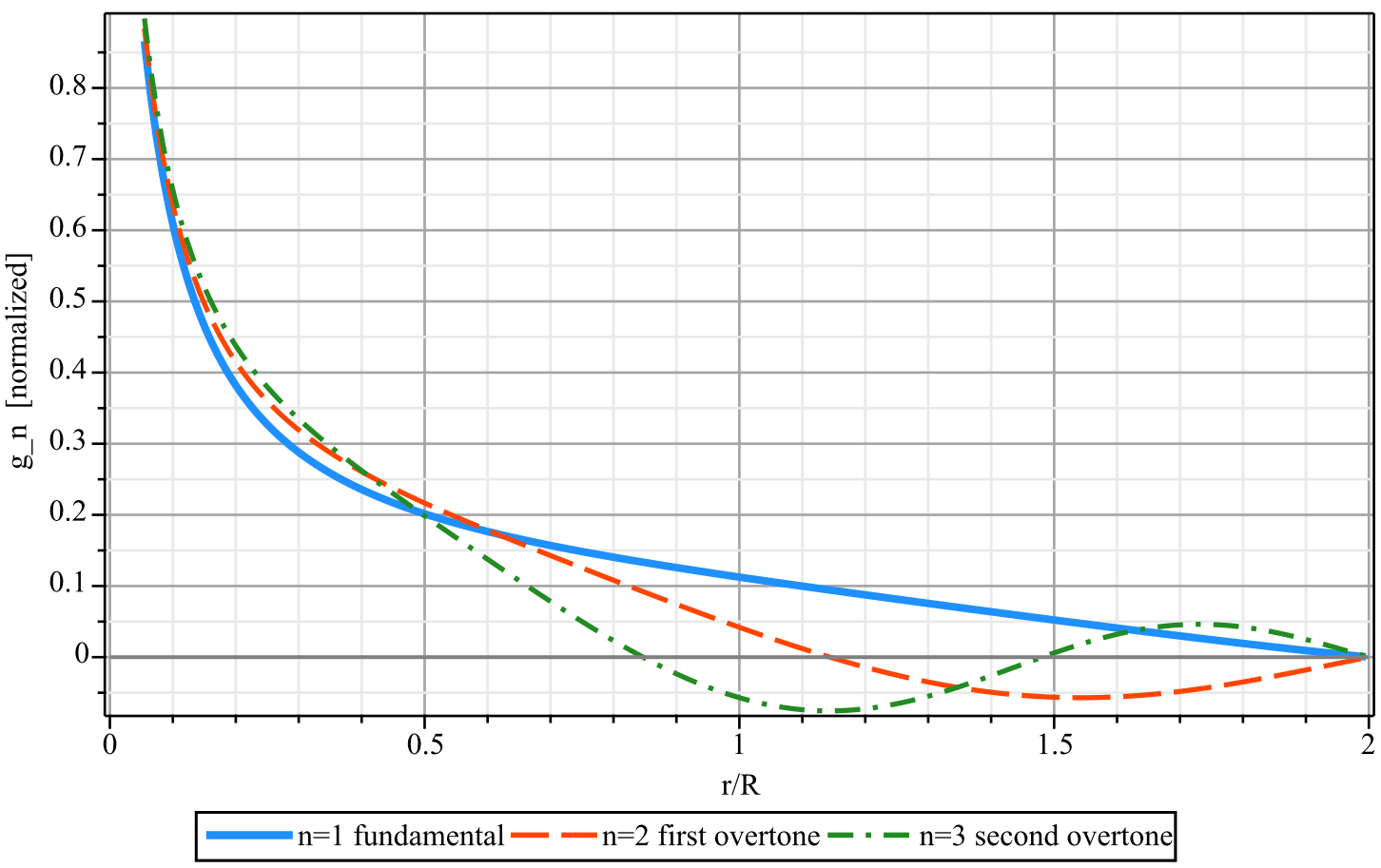

0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
1.6
1.8
2.0
x
n=2 first overtone


0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
1.6
1.8
2.0
x
n=3 second overtone


1
0.5
0
−0.5
1
0.5
1
1.5
2
x


Airy Eigenstates g_n(r) of (R/r)*Laplacian(g)=-k_n^2*g
g(2R)=0, peak-normalized
g_n [normalized]
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
0
0.5
1
1.5
2
r/R
n=1 fundamental
n=2 first overtone
n=3 second overtone

"Plot 2 done."

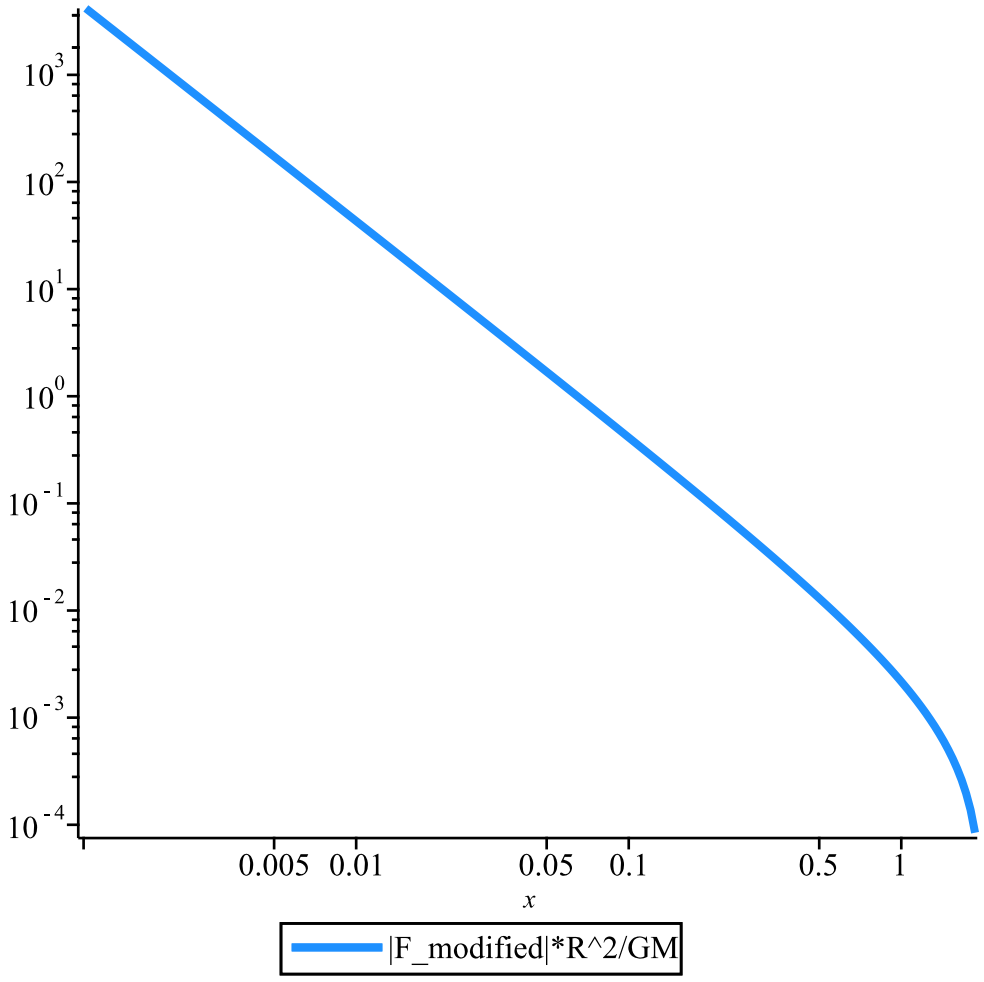


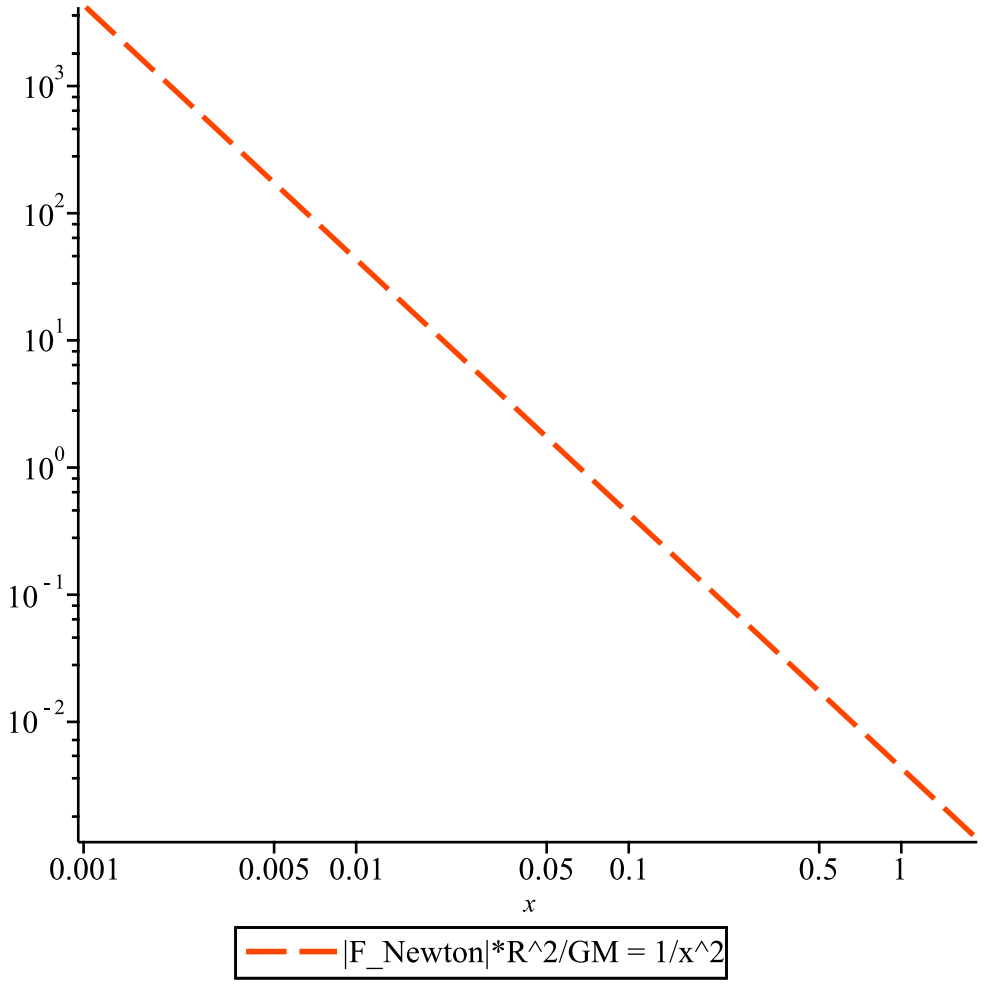
"=="

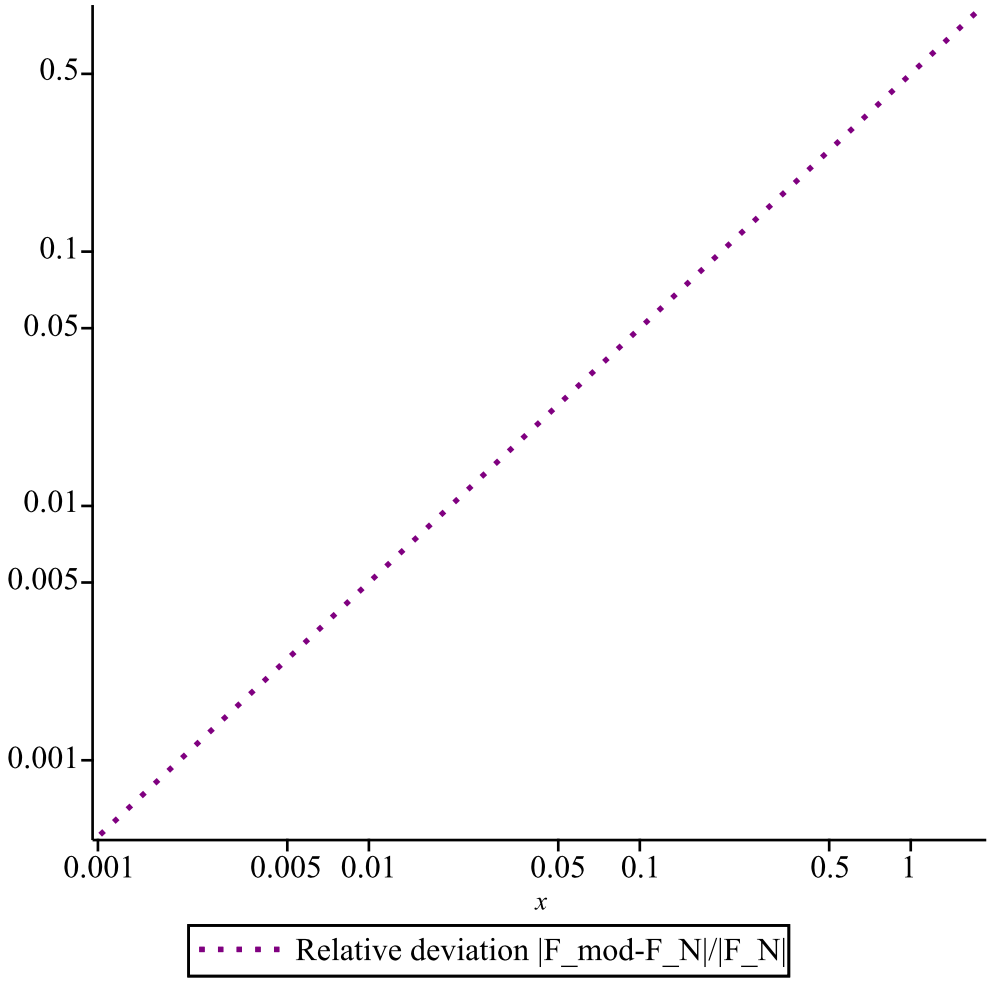
" PLOT 3: Modified Force vs Newton (log-log)"

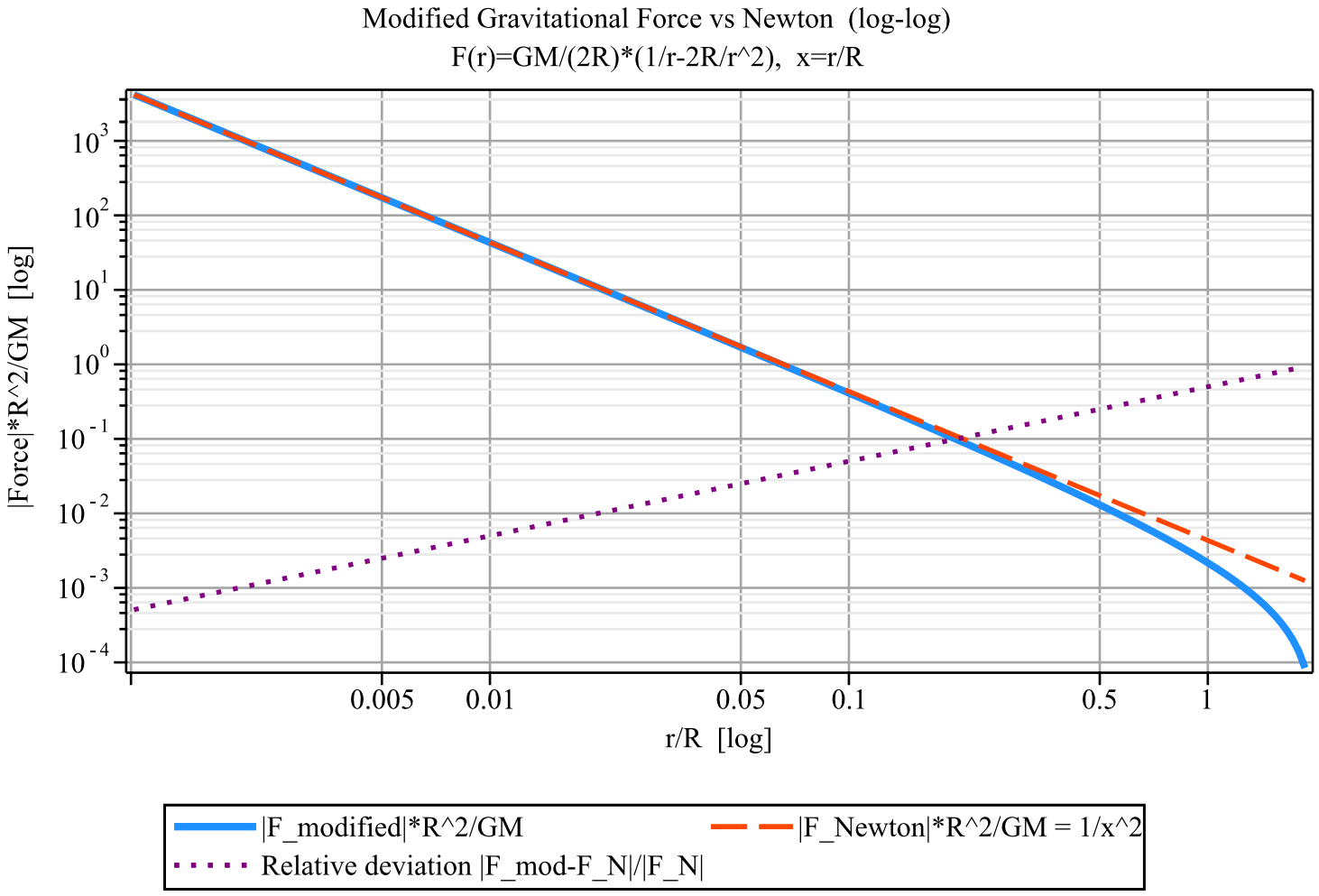
"=="

GM_R2 := 0.004337044408

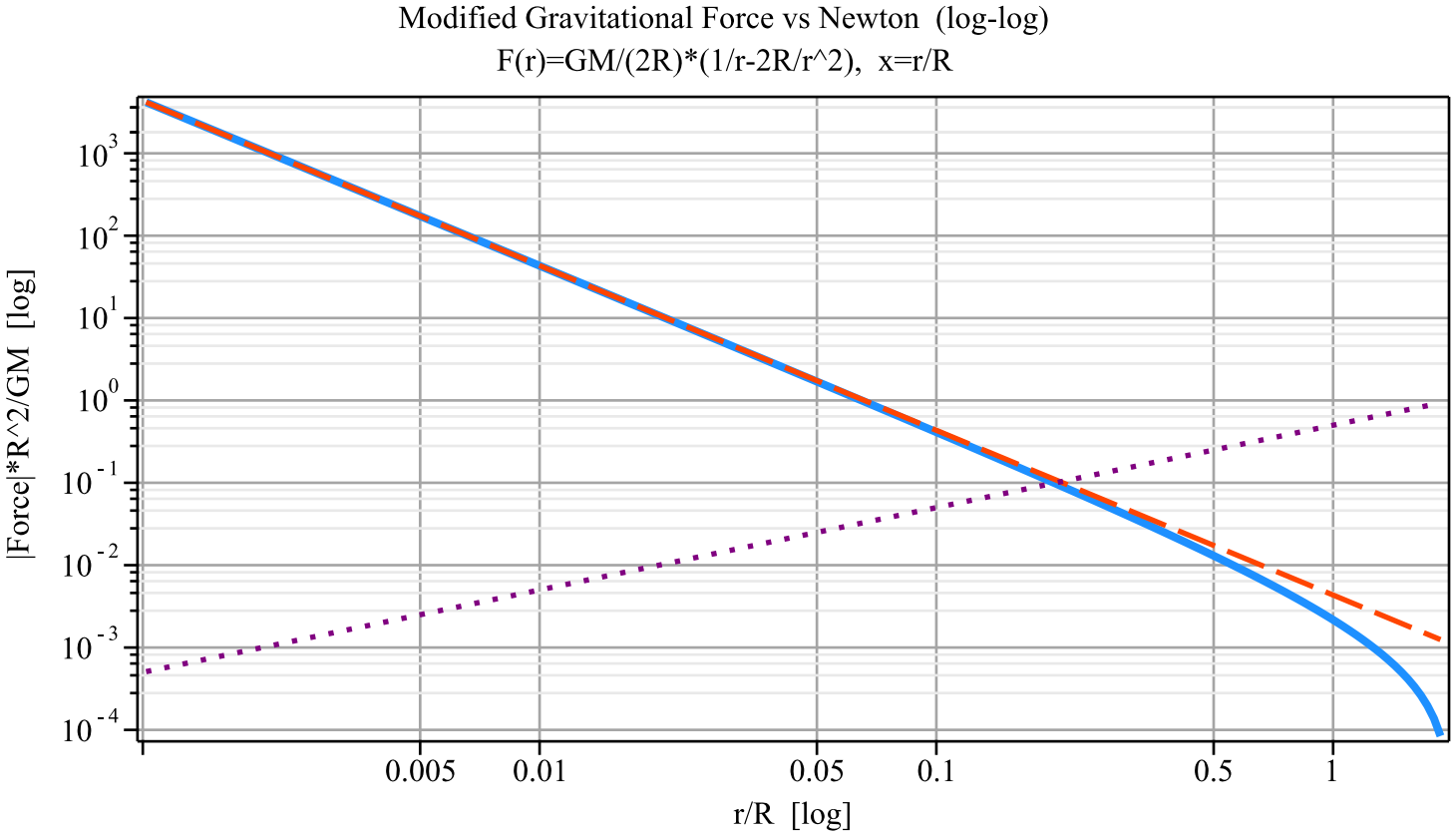

10^3
10^2
10^1
10^0
10^-1
10^-2
10^-3
10^-4
0.005
0.01
0.05
0.1
0.5
1
x
|F_modified|*R^2/GM


10^3
10^2
10^1
10^0
10^-1
10^-2
0.001
0.005
0.01
0.05
0.1
0.5
1
x
|F_Newton|*R^2/GM = 1/x^2


0.5
0.1
0.05
0.01
0.005
0.001
0.001
0.005
0.01
0.05
0.1
0.5
1
x
Relative deviation |F_mod-F_N|/|F_N|


Modified Gravitational Force vs Newton (log-log)
F(r)=GM/(2R)*(1/r-2R/r^2), x=r/R
|Force|*R^2/GM [log]
10^3
10^2
10^1
10^0
10^-1
10^-2
10^-3
10^-4
0.005
0.01
0.05
0.1
0.5
1
r/R [log]
|F_modified|*R^2/GM
|F_Newton|*R^2/GM = 1/x^2
Relative deviation |F_mod-F_N|/|F_N|

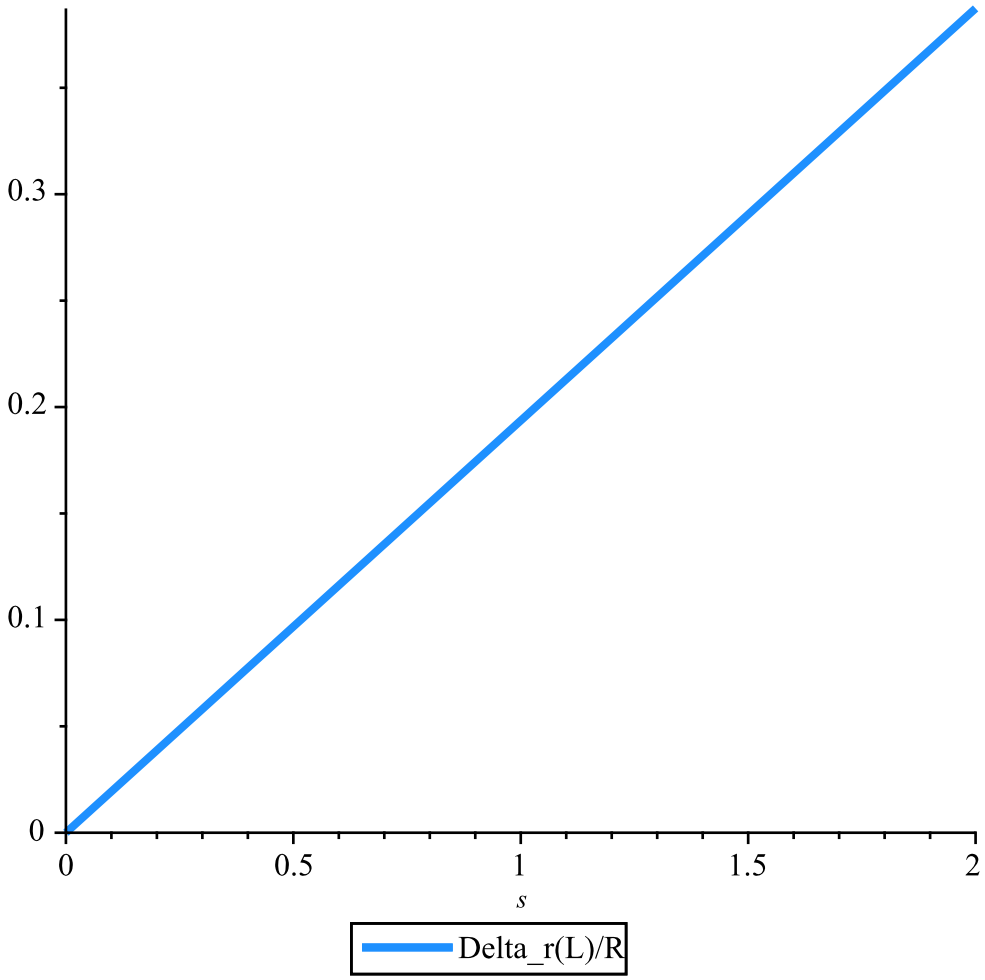
"Plot 3 done."

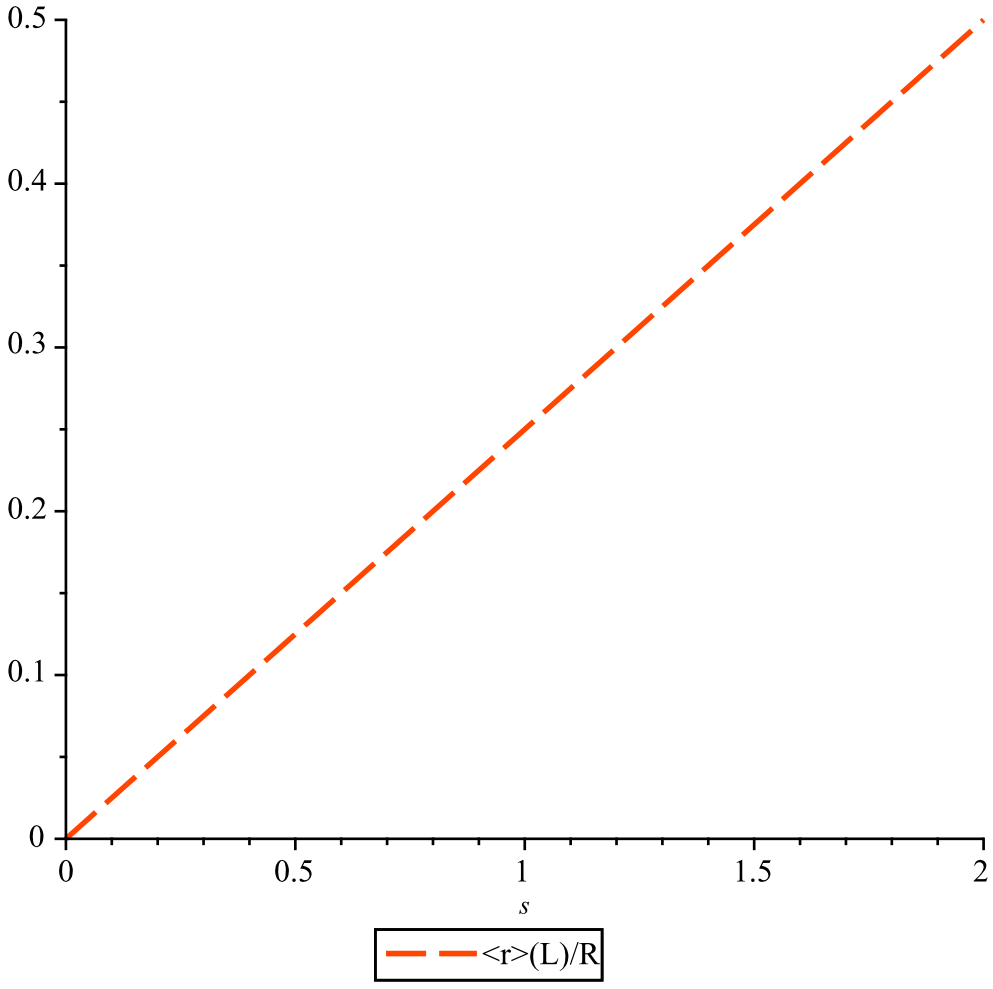


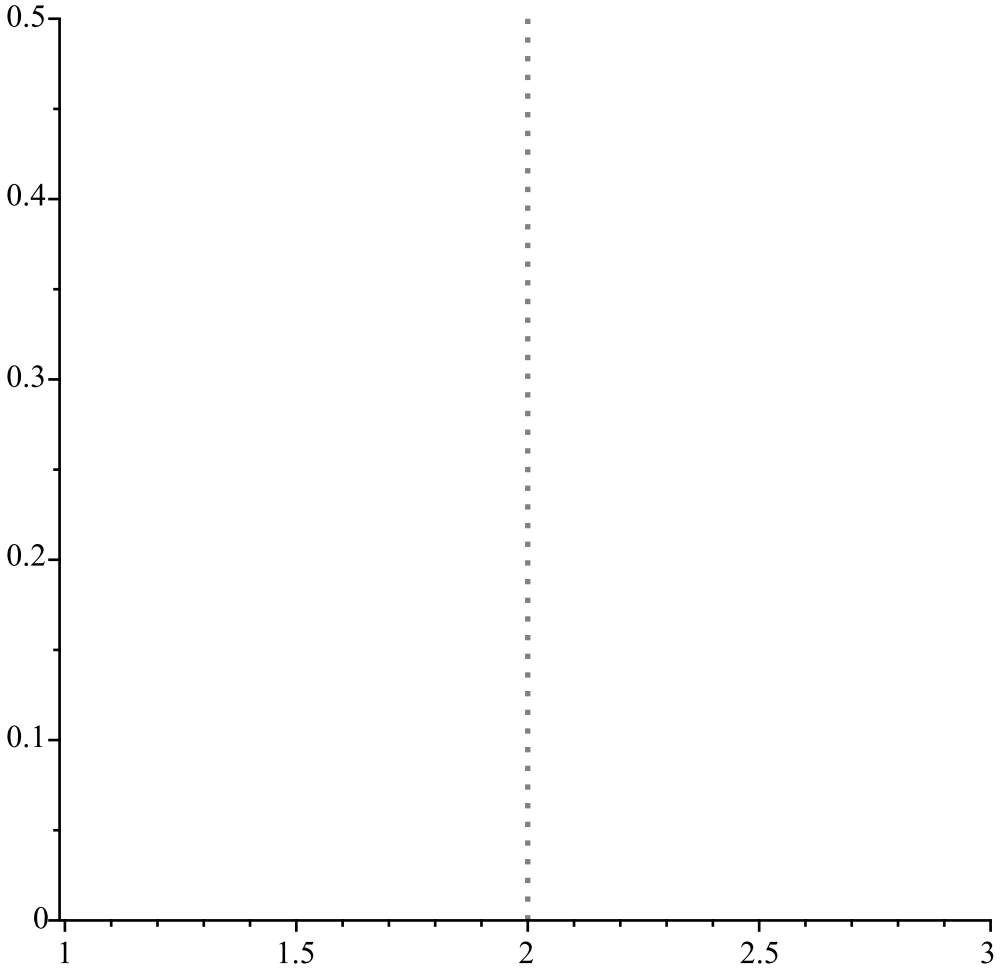
"==========="

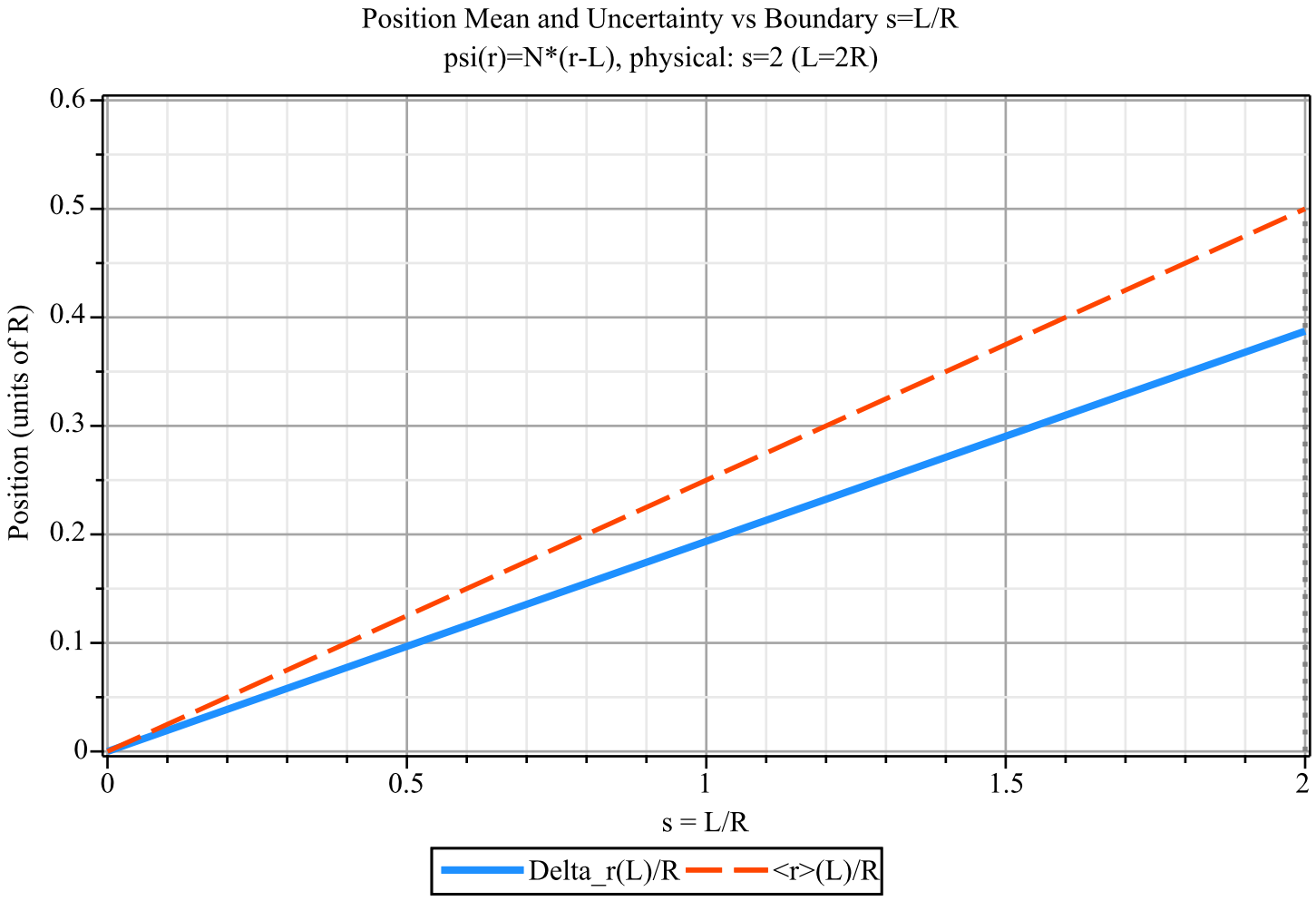
" PLOT 4: Delta_r and <r> vs Boundary s=L/R"

"==========="

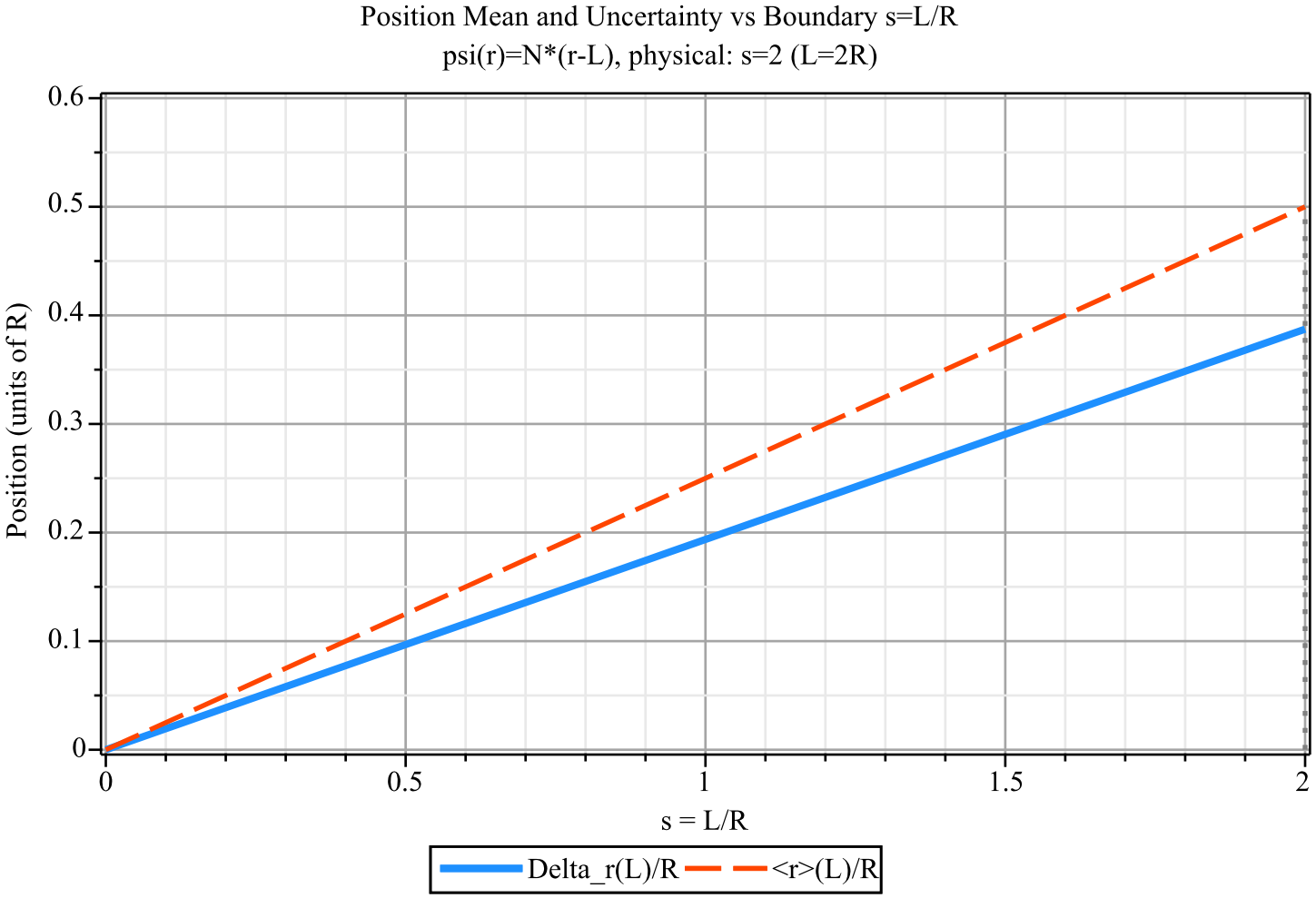

0.3
0.2
0.1
0
0
0.5
1
1.5
2
s
Delta_r(L)/R


0.5
0.4
0.3
0.2
0.1
0
0
0.5
1
1.5
2
s
<r>(L)/R


0.5
0.4
0.3
0.2
0.1
0
1
1.5
2
2.5
3


Position Mean and Uncertainty vs Boundary s=L/R
psi(r)=N*(r-L), physical: s=2 (L=2R)
Position (units of R)
0.6
0.5
0.4
0.3
0.2
0.1
0
0
0.5
1
1.5
2
s = L/R
Delta_r(L)/R
<r>(L)/R

"Plot 4 done."



"=="

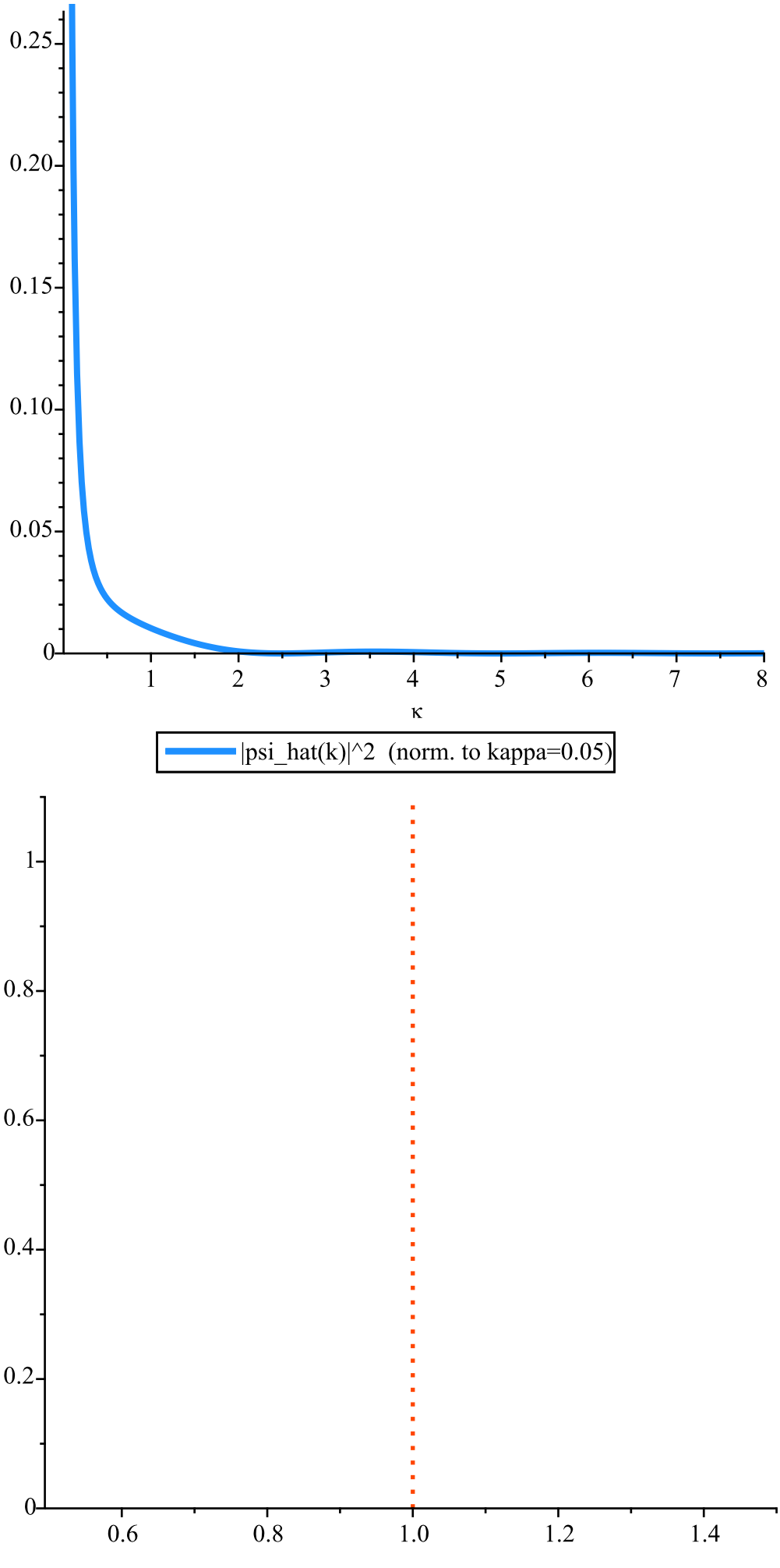
" PLOT 5: Momentum Space |psi_hat(k)|^2"

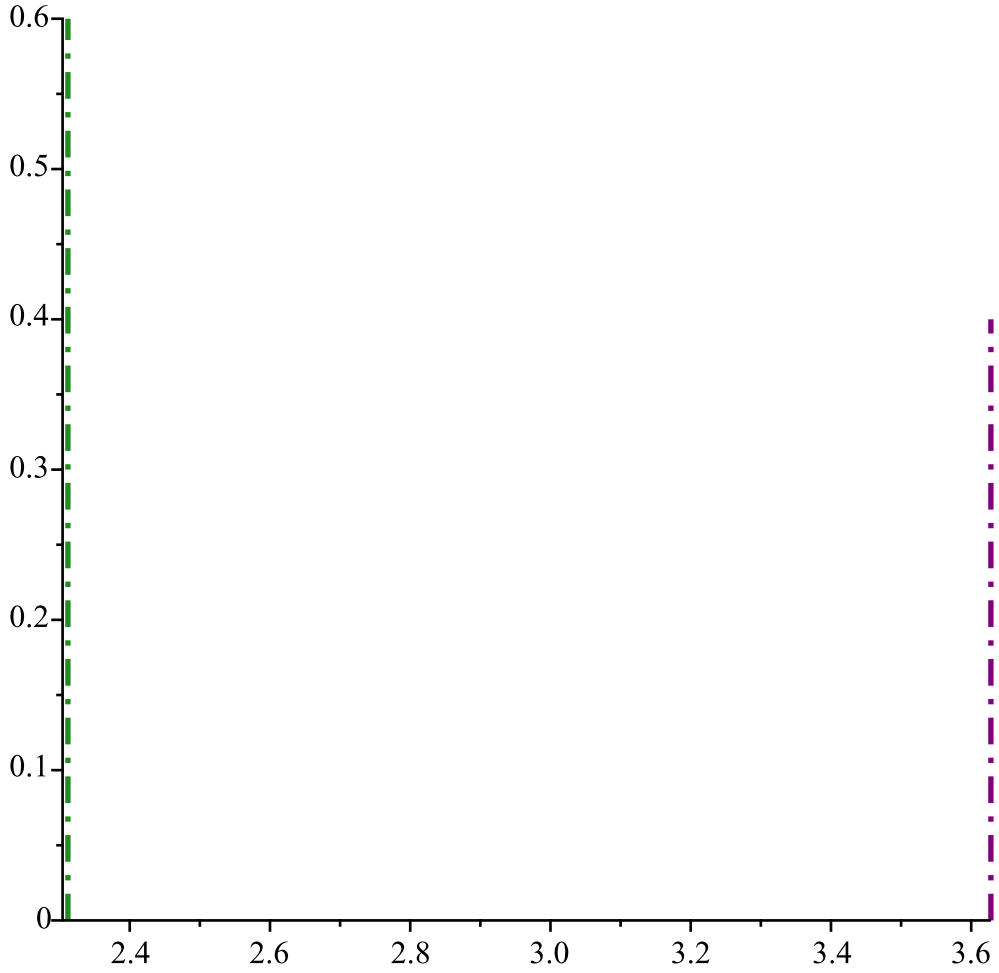
"=="

$$Nv := 7.881726924 \times 10^{-35}$$

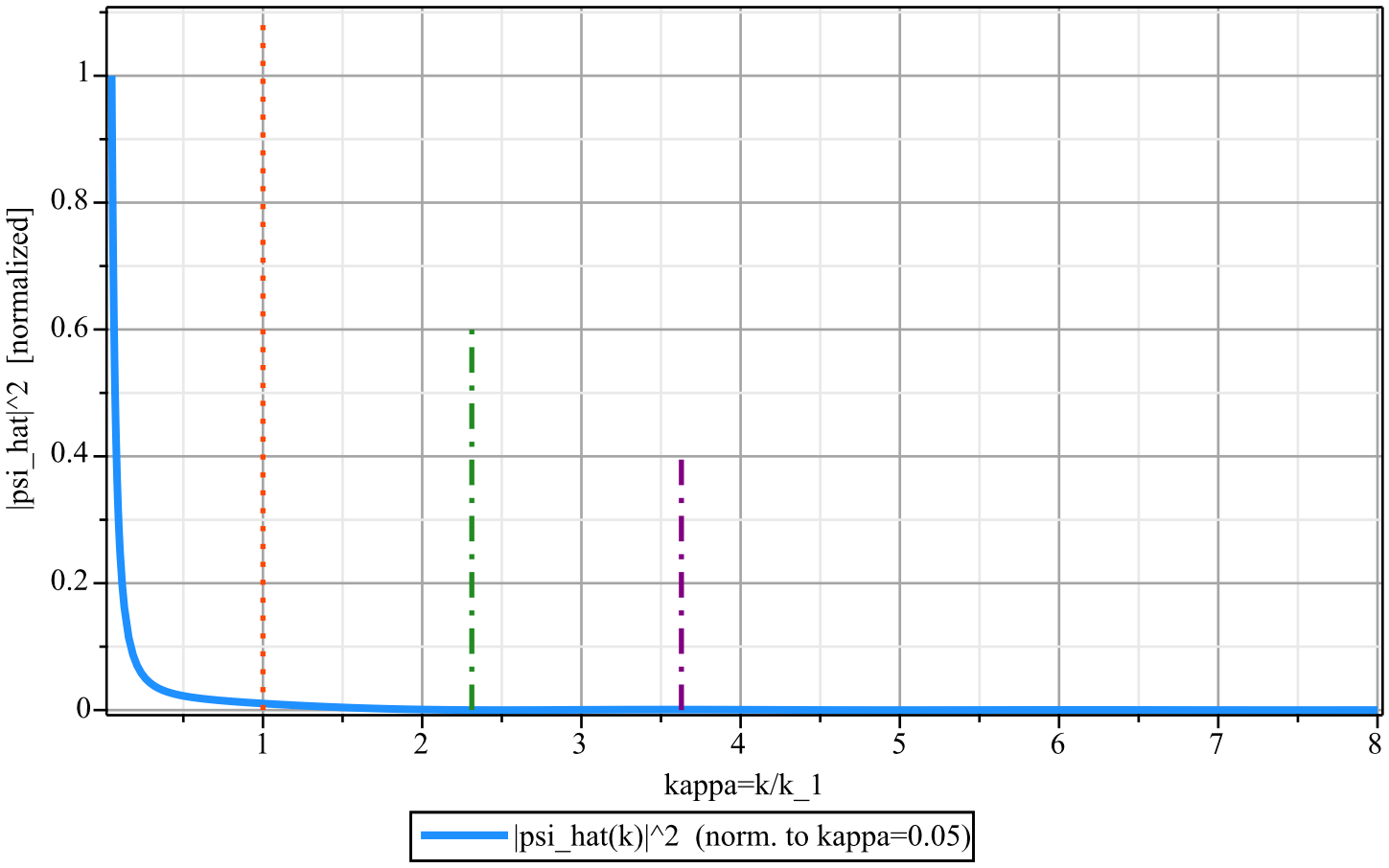
$$Rv := 3.9228 \times 10^{22}$$

$$k1v := 3.222222512 \times 10^{-23}$$

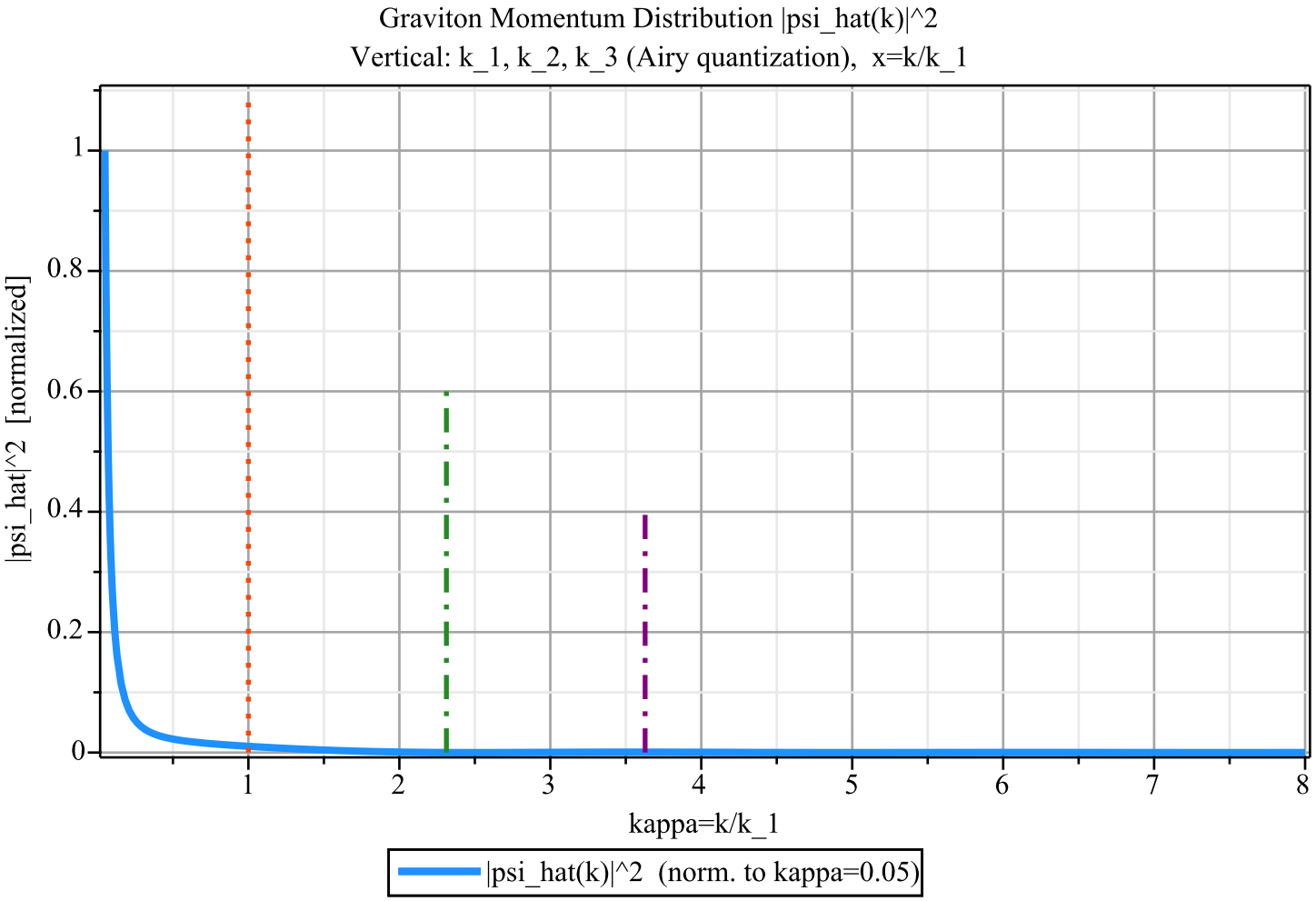

0.25
0.20
0.15
0.10
0.05
0
1
2
3
4
5
6
7
8
κ
|psi_hat(k)|^2 (norm. to kappa=0.05)
1
0.8
0.6
0.4
0.2
0
0.6
0.8
1.0
1.2
1.4



Graviton Momentum Distribution |psi_hat(k)|^2
Vertical: k_1, k_2, k_3 (Airy quantization), x=k/k_1



"Plot 5 done."

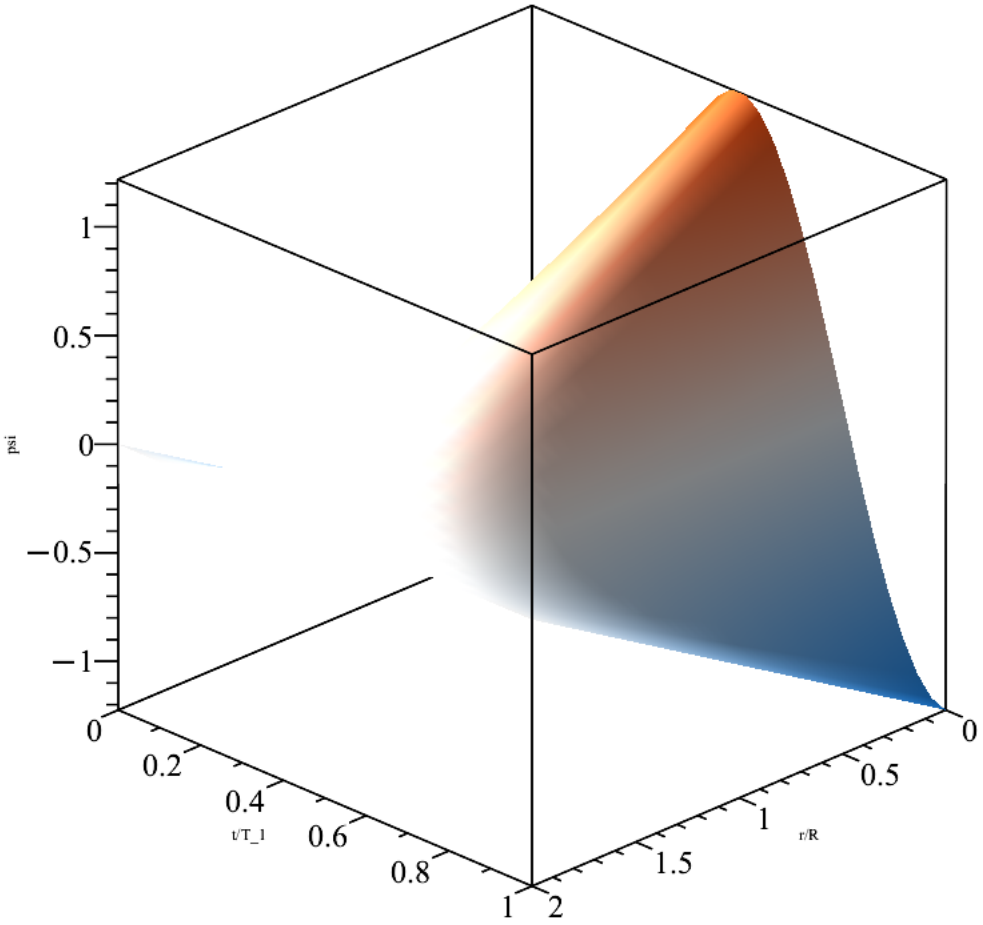


"=="

" PLOT 6: Spacetime Evolution psi(r,t) [3D]"

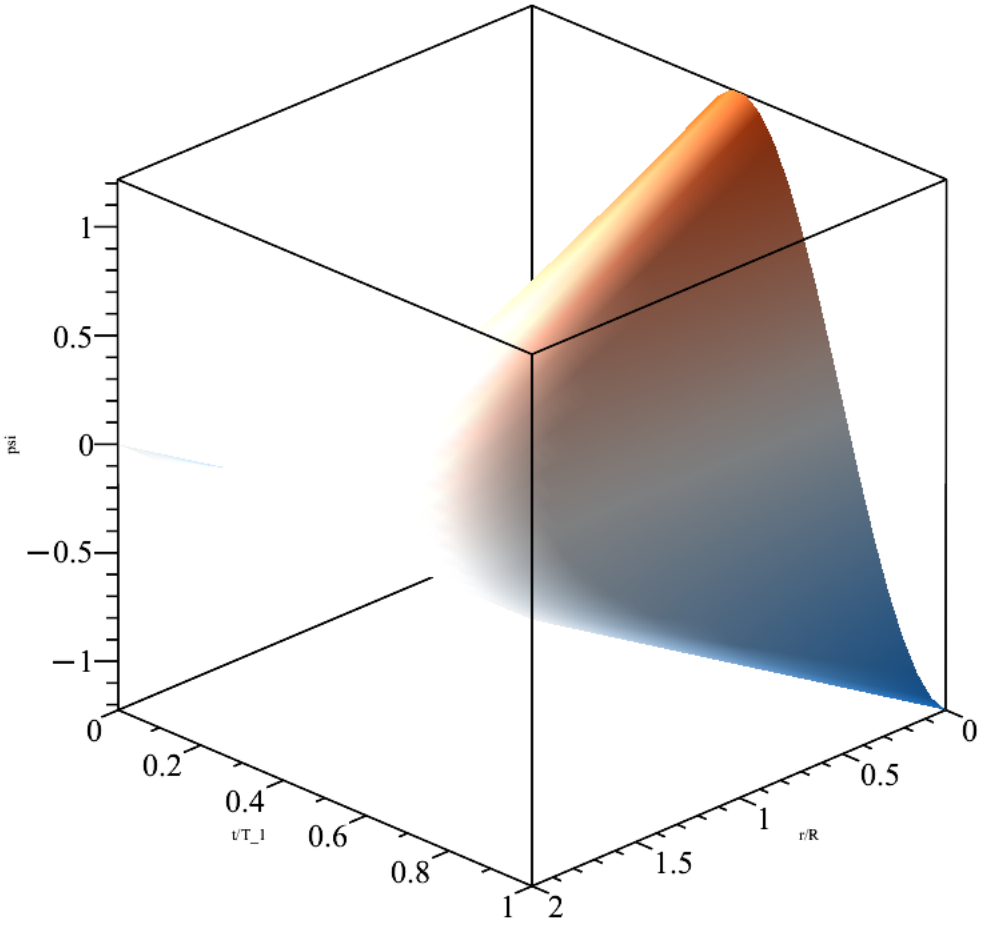
"=="

Graviton Wave Function psi(r,t)=N*(r-2R)*cos(omega1*t)
x=r/R in [0,2], tau=t/T_1



"Plot 6 done."

Graviton Wave Function psi(r,t)=N*(r-2R)*cos(omega1*t)
x=r/R in [0,2], tau=t/T_1

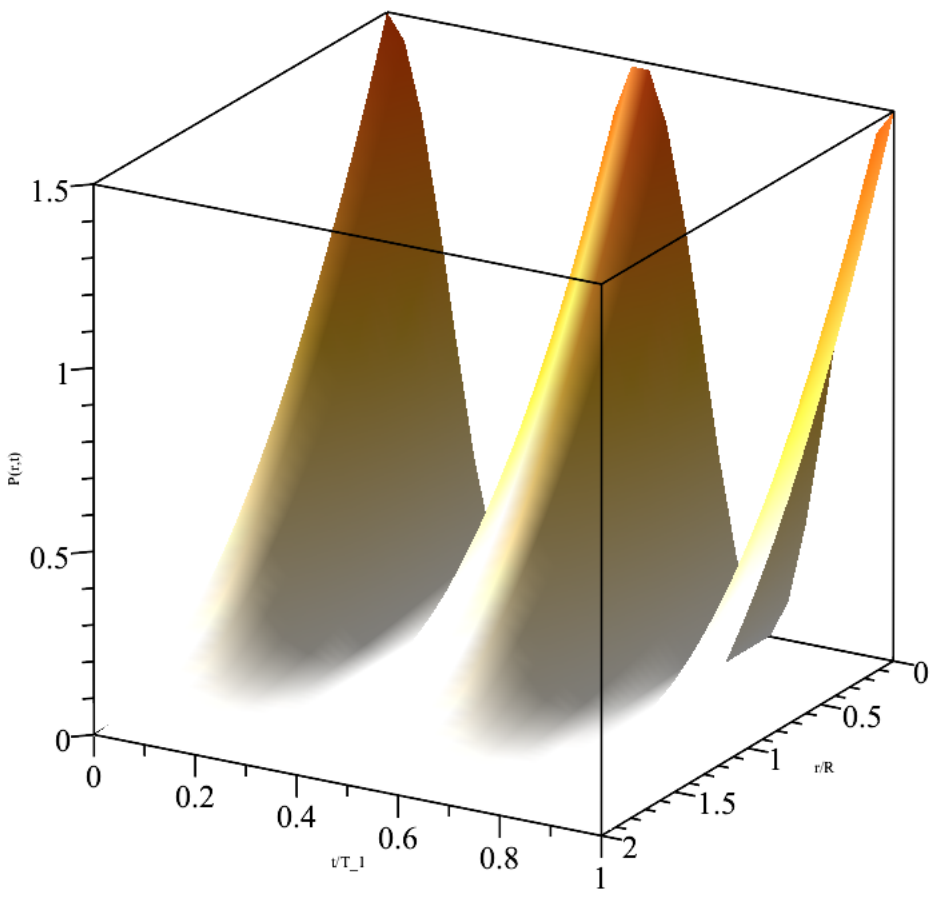


"==========="

" PLOT 7: Probability Density P(r,t)=|psi|^2 [3D]"

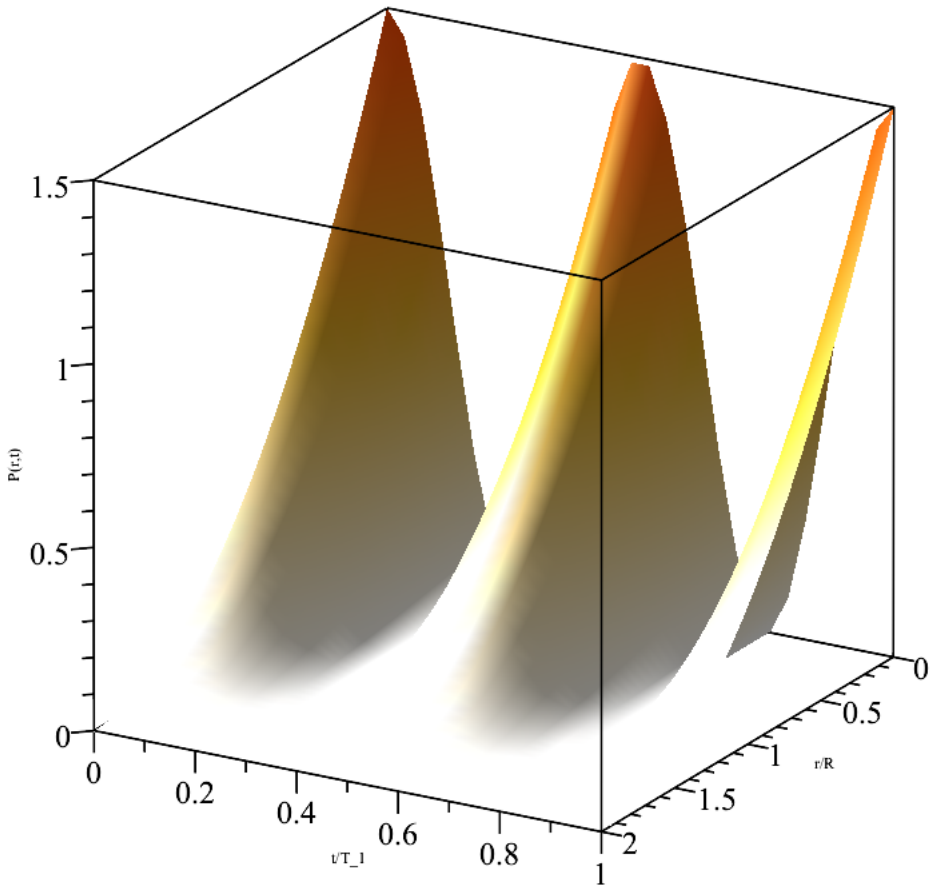
"==========="

Graviton Probability Density P(r,t)=|psi|^2
P=(3/8)*(r/R-2)^2*cos^2(omega_1*t)



"Plot 7 done."

Graviton Probability Density P(r,t)=|psi|^2
P=(3/8)*(r/R-2)^2*cos^2(omega_1*t)



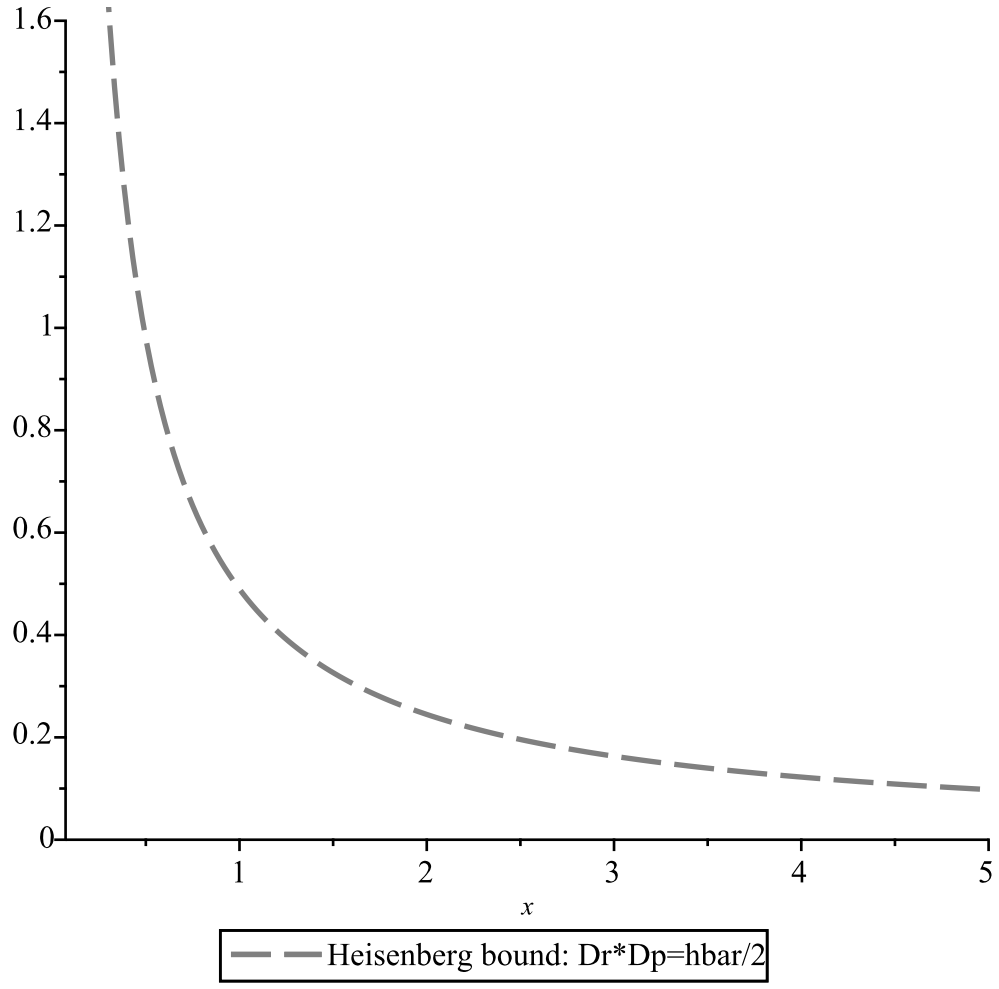
"=="

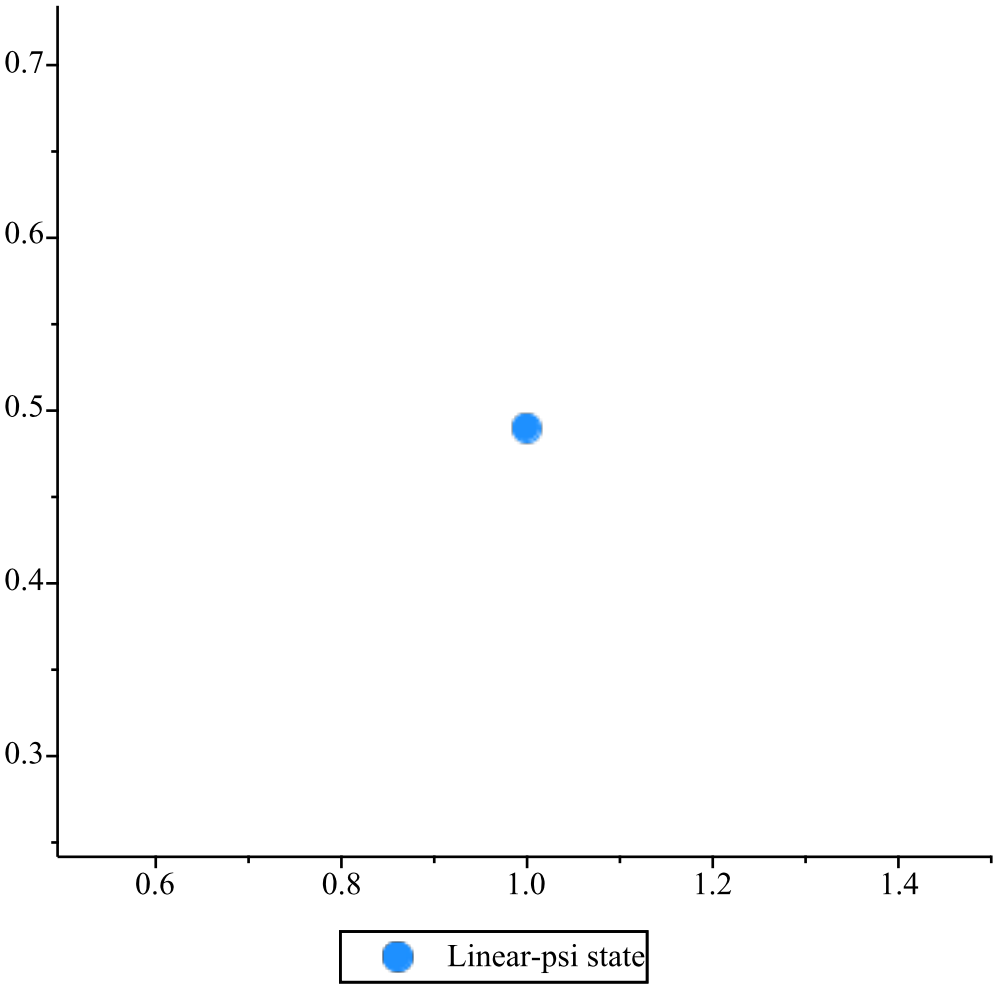
" PLOT 8: Heisenberg Phase-Space Diagram"

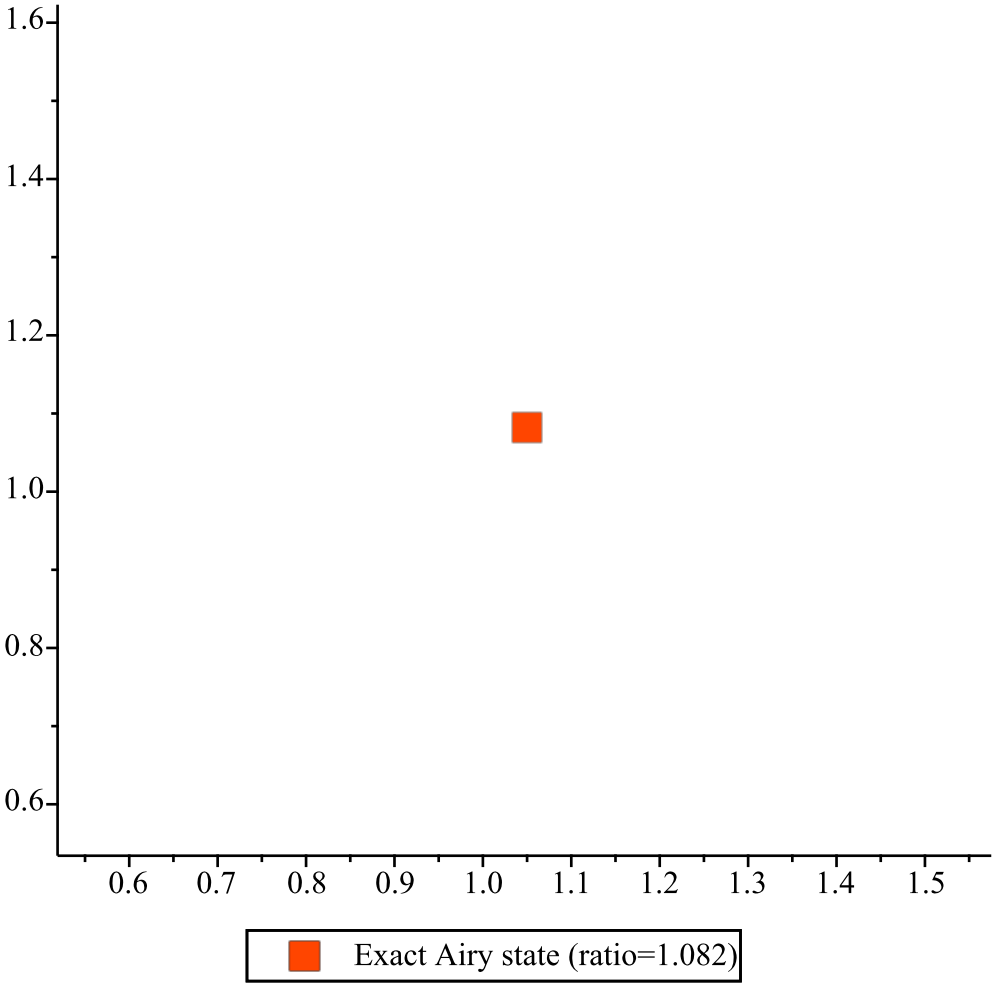
"=="

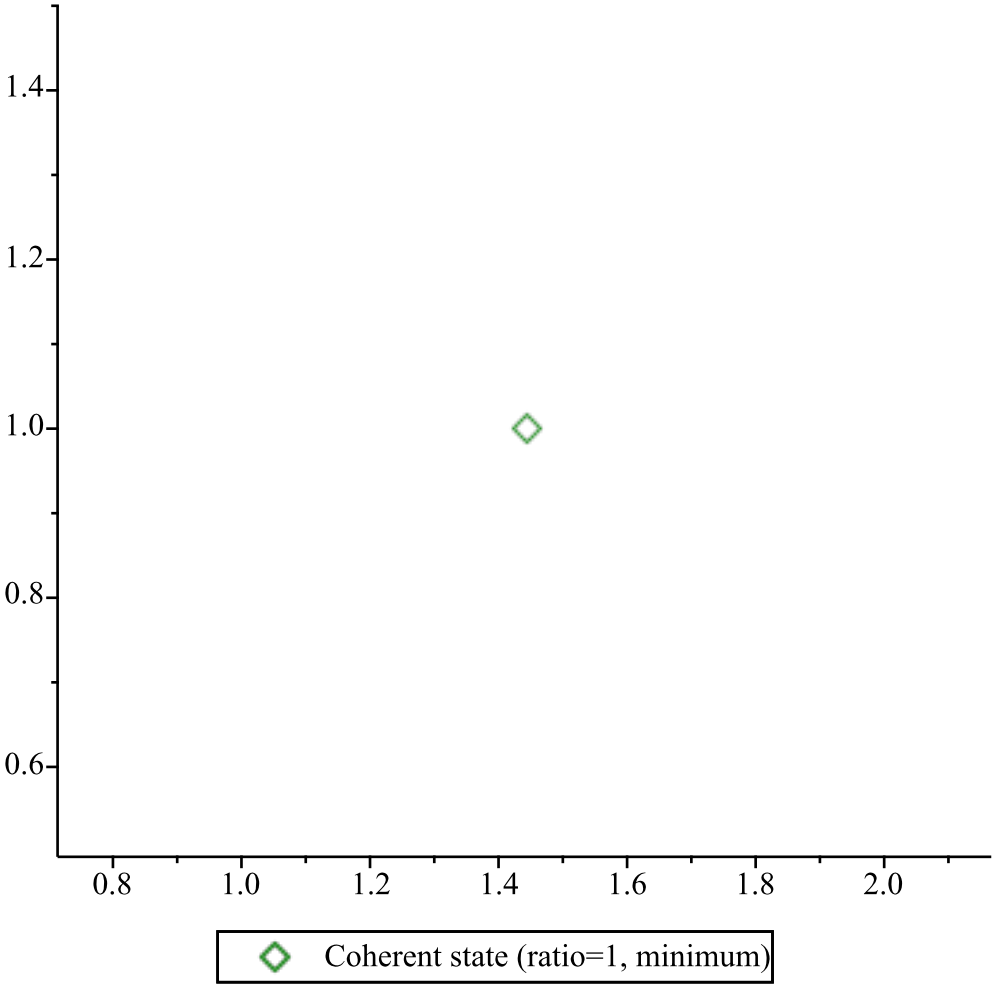
ratio_val := 0.4895503030

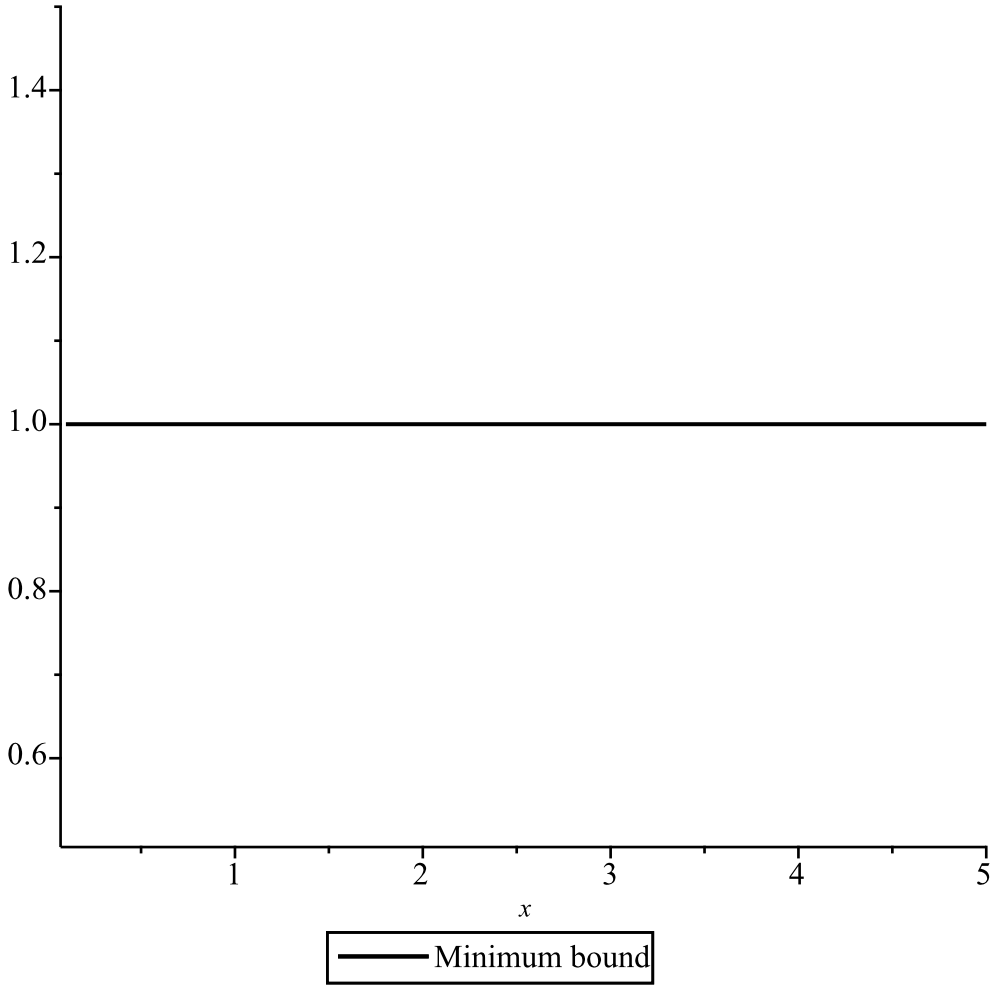
sigma_r_norm := 1.444400661

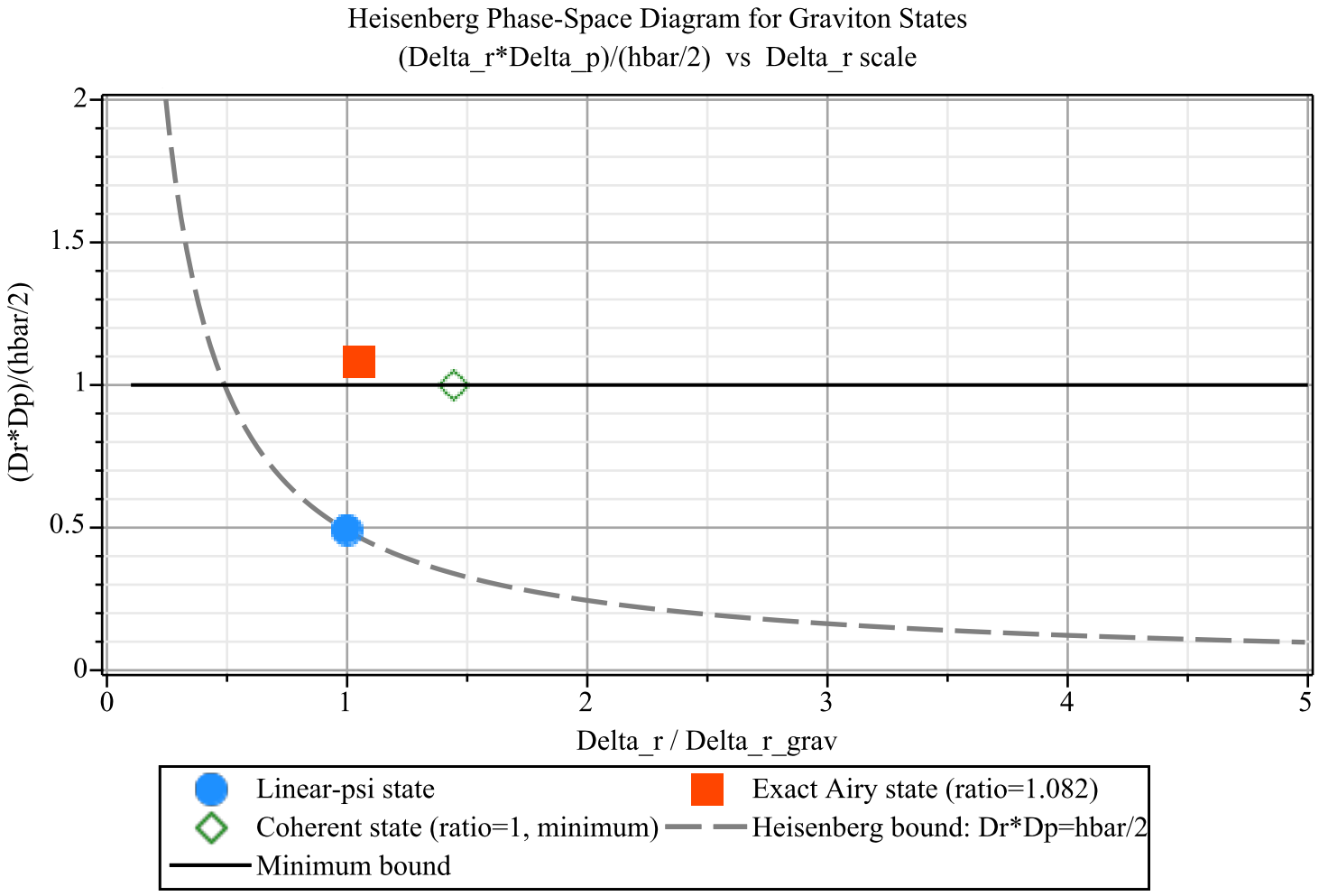

1.6
1.4
1.2
1
0.8
0.6
0.4
0.2
0
1
2
3
4
5
x
Heisenberg bound: Dr*Dp=hbar/2


0.7
0.6
0.5
0.4
0.3
0.6
0.8
1.0
1.2
1.4
Linear-psi state

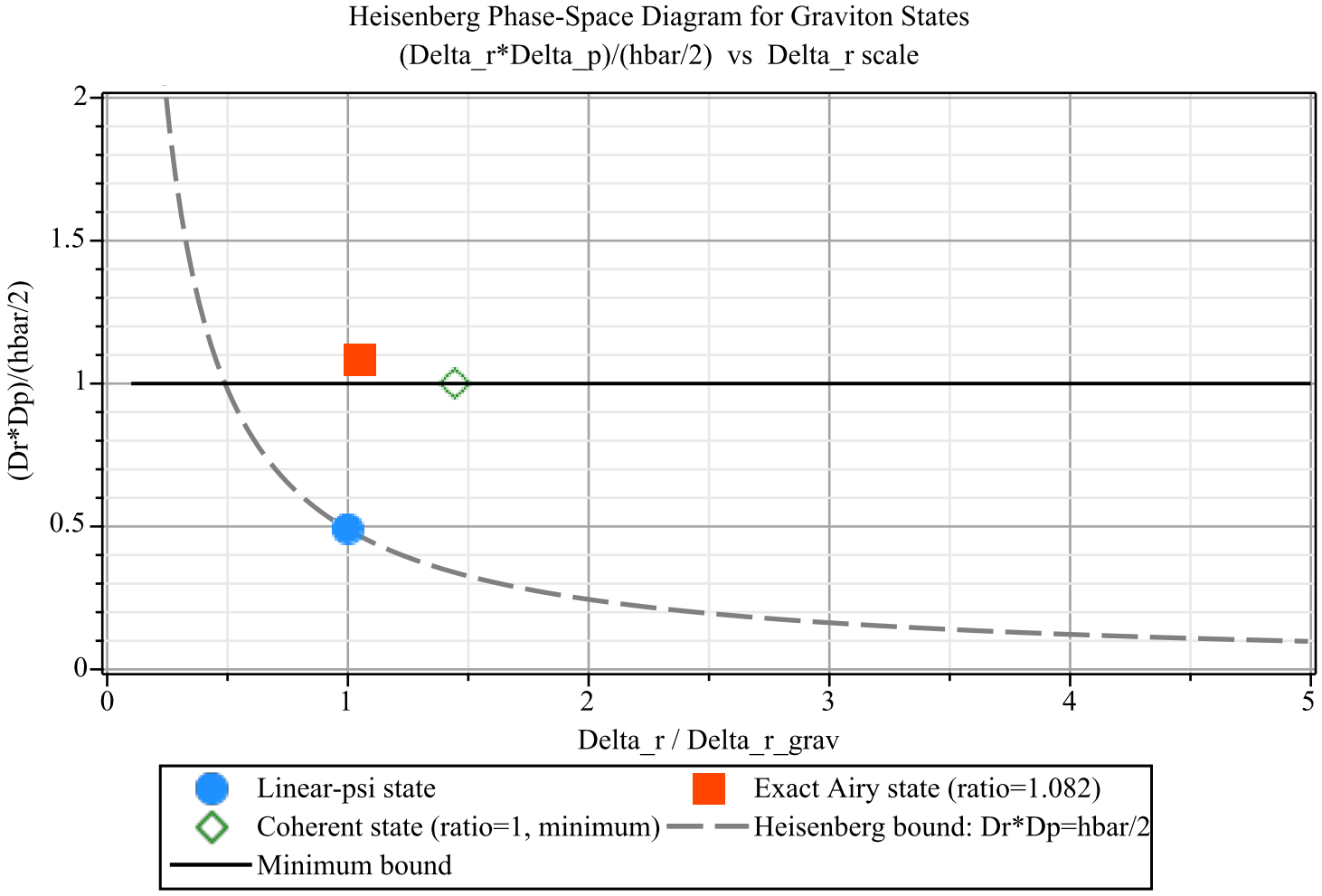

1.6
1.4
1.2
1.0
0.8
0.6
0.6 0.7 0.8 0.9 1.0 1.1 1.2 1.3 1.4 1.5
Exact Airy state (ratio=1.082)


1.4
1.2
1.0
0.8
0.6
0.8 1.0 1.2 1.4 1.6 1.8 2.0
Coherent state (ratio=1, minimum)


1.4
1.2
1.0
0.8
0.6
1
2
3
4
5
x
Minimum bound


Heisenberg Phase-Space Diagram for Graviton States
(Delta_r*Delta_p)/(hbar/2) vs Delta_r scale
(Dr*Dp)/(hbar/2)
2
1.5
1
0.5
0
0
1
2
3
4
5
Delta_r / Delta_r_grav
Linear-psi state
Exact Airy state (ratio=1.082)
Coherent state (ratio=1, minimum)
Heisenberg bound: Dr*Dp=hbar/2
Minimum bound

"Plot 8 done."



"=="

" PLOT 9: Graviton Energy Spectrum (First 10 Modes)"

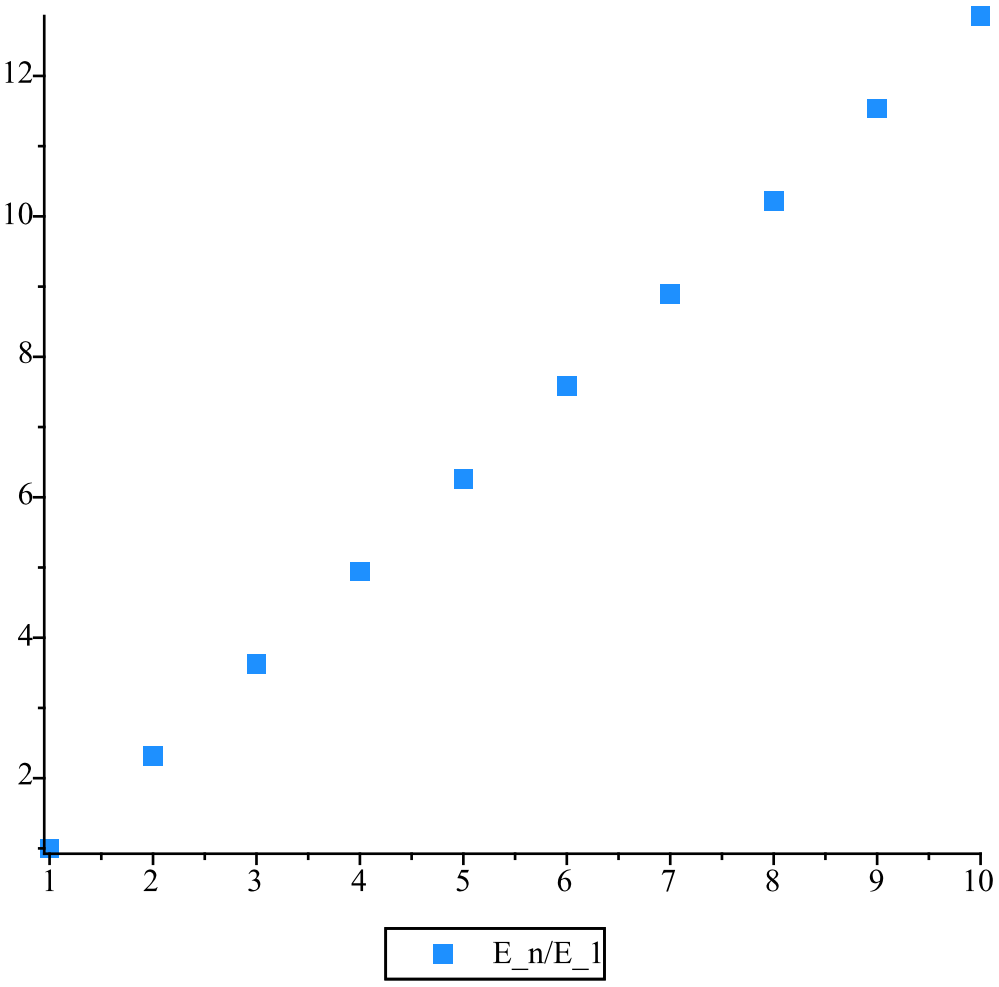
"=="

$airy_zeros := [-2.33810741, -4.08794944, -5.52055983, -6.78670809, -7.94413359, -9.02265085, -10.0401743, -11.0085243, -11.9360155, -12.8287768]$

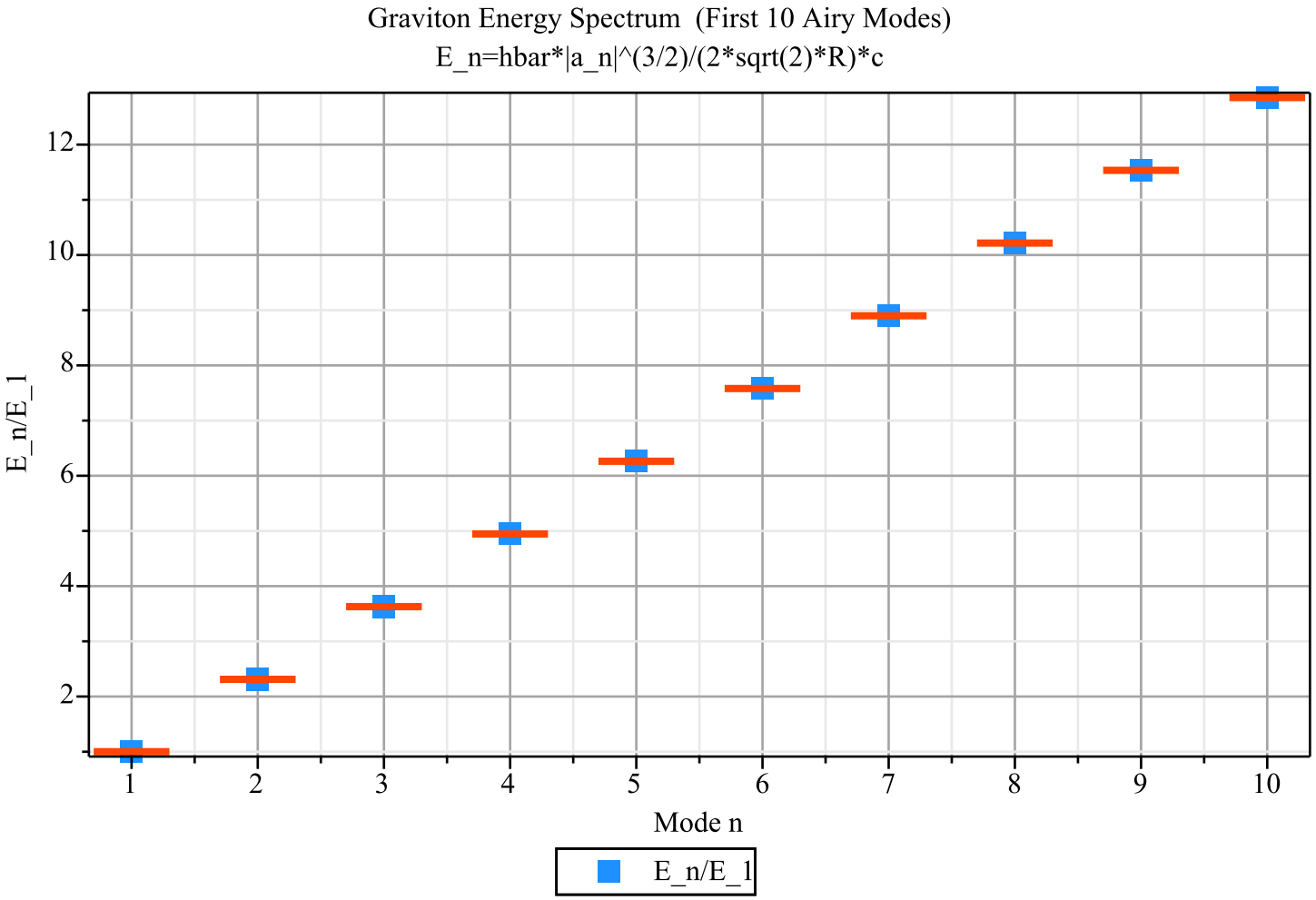
$E_raw := [1.018712518 \times 10^{-48}, 2.355121022 \times 10^{-48}, 3.695980206 \times 10^{-48}, 5.037832340 \times 10^{-48}, 6.380060010 \times 10^{-48}, 7.722469165 \times 10^{-48}, 9.064979660 \times 10^{-48}, 1.040755258 \times 10^{-47}, 1.175016639 \times 10^{-47}, 1.309280890 \times 10^{-47}]$

$En_norm := [1.000000000, 2.311860295, 3.628089516, 4.945293447, 6.262866017, 7.580616738, 8.898466937, 10.21637842, 11.53433003, 12.85230982]$

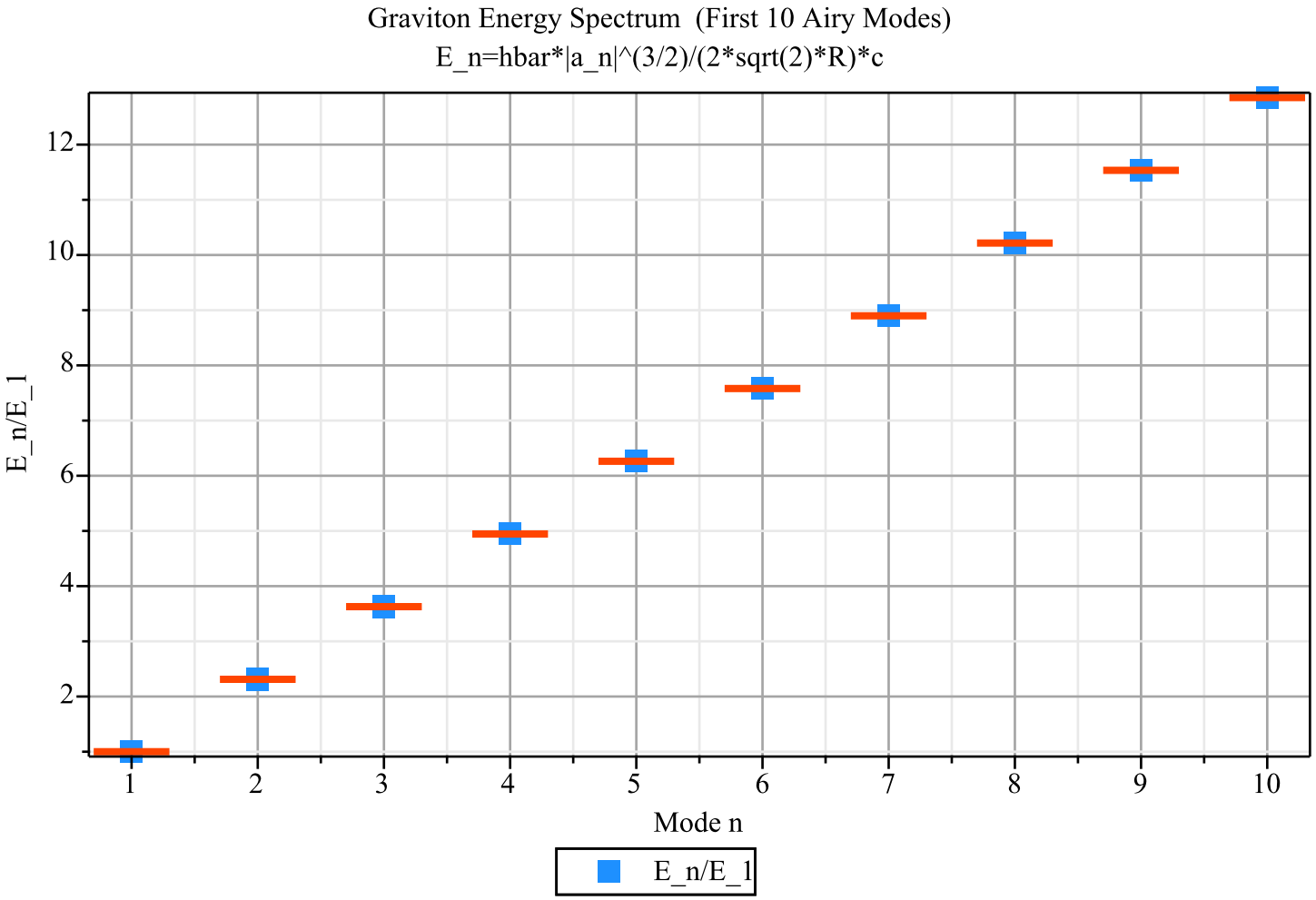
"E_n/E_1 =", [1.000000000, 2.311860295, 3.628089516, 4.945293447, 6.262866017, 7.580616738, 8.898466937, 10.21637842, 11.53433003, 12.85230982]



p9bars := [*CURVES*([[0.7, 1.000000000], [1.3, 1.000000000]], *COLOUR*(*RGB*, 1.00000000, 0.27058824, 0.), *THICKNESS*(3)), *CURVES*([[1.7, 2.311860295], [2.3, 2.311860295]], *COLOUR*(*RGB*, 1.00000000, 0.27058824, 0.), *THICKNESS*(3)), *CURVES*([[2.7, 3.628089516], [3.3, 3.628089516]], *COLOUR*(*RGB*, 1.00000000, 0.27058824, 0.), *THICKNESS*(3)), *CURVES*([[3.7, 4.945293447], [4.3, 4.945293447]], *COLOUR*(*RGB*, 1.00000000, 0.27058824, 0.), *THICKNESS*(3)), *CURVES*([[4.7, 6.262866017], [5.3, 6.262866017]], *COLOUR*(*RGB*, 1.00000000, 0.27058824, 0.), *THICKNESS*(3)), *CURVES*([[5.7, 7.580616738], [6.3, 7.580616738]], *COLOUR*(*RGB*, 1.00000000, 0.27058824, 0.), *THICKNESS*(3)), *CURVES*([[6.7, 8.898466937], [7.3, 8.898466937]], *COLOUR*(*RGB*, 1.00000000, 0.27058824, 0.), *THICKNESS*(3)), *CURVES*([[7.7, 10.21637842], [8.3, 10.21637842]], *COLOUR*(*RGB*, 1.00000000, 0.27058824, 0.), *THICKNESS*(3)), *CURVES*([[8.7, 11.53433003], [9.3, 11.53433003]], *COLOUR*(*RGB*, 1.00000000, 0.27058824, 0.), *THICKNESS*(3)), *CURVES*([[9.7, 12.85230982], [10.3, 12.85230982]], *COLOUR*(*RGB*, 1.00000000, 0.27058824, 0.), *THICKNESS*(3))]


Graviton Energy Spectrum (First 10 Airy Modes)
E_n=hbar*|a_n|^(3/2)/(2*sqrt(2)*R)*c
E_n/E_1
Mode n
E_n/E_1

"Plot 9 done."



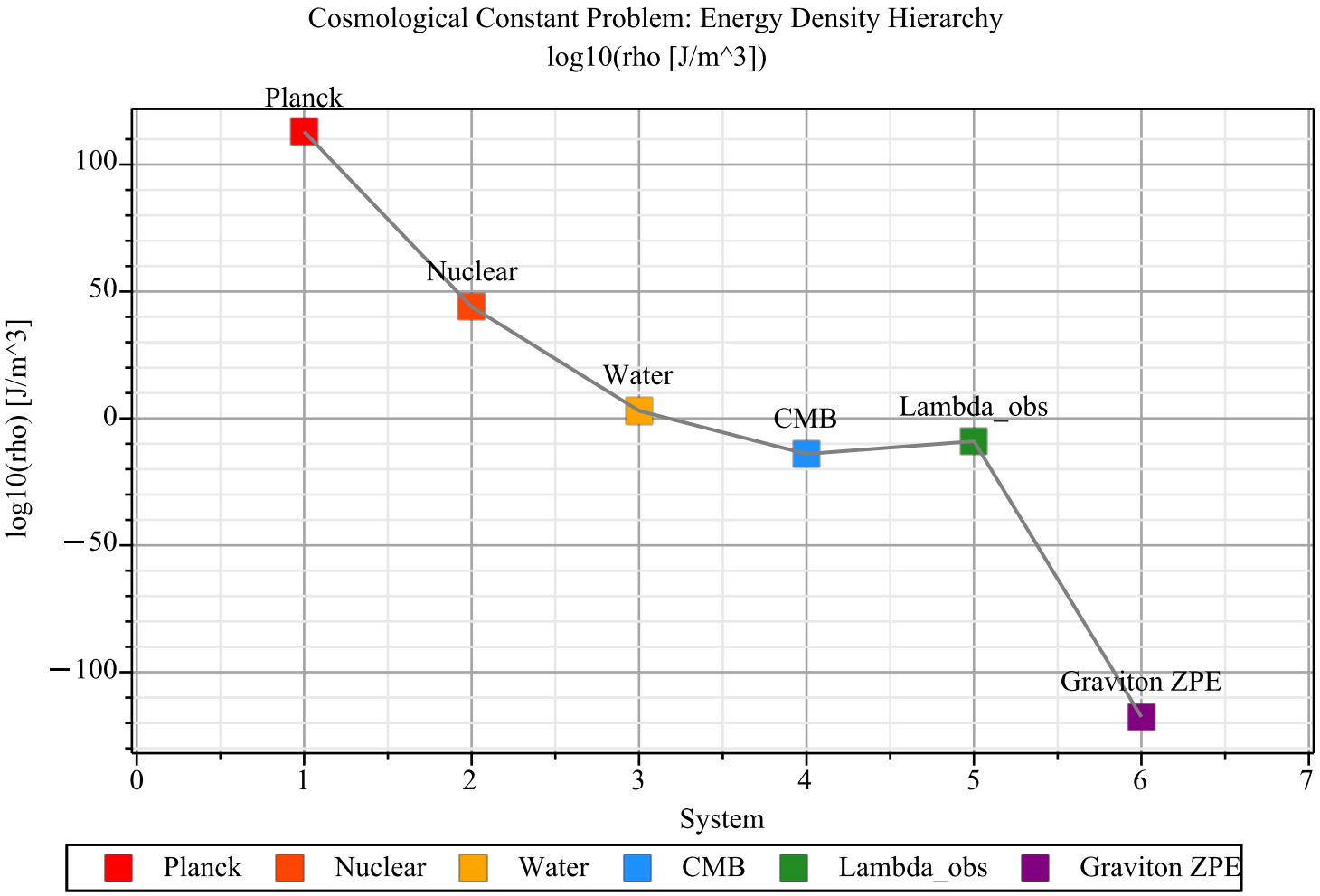
"=="

" PLOT 10: Energy Density Hierarchy"

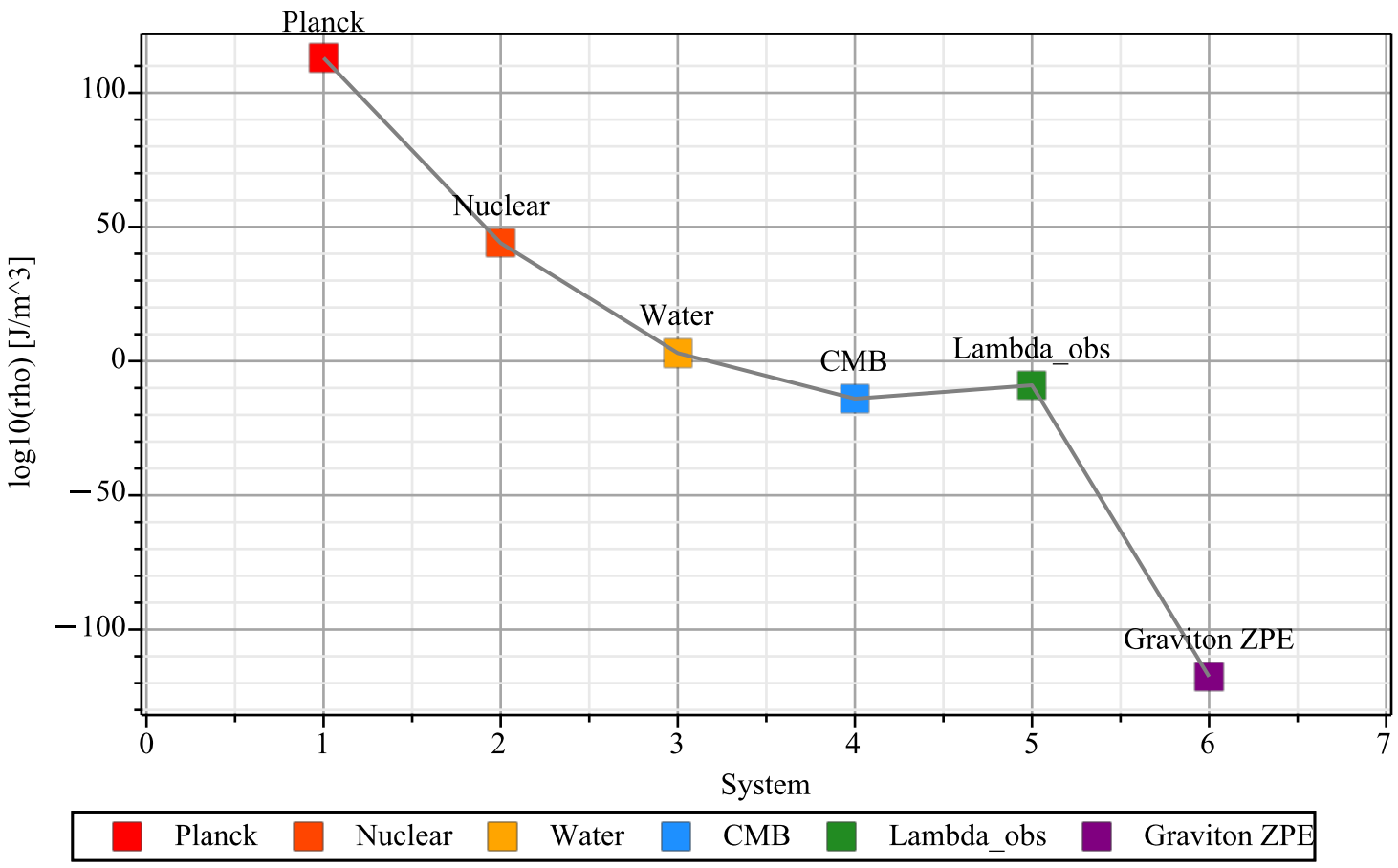
"=="

rho_log10v := [113.0, 44.0, 3.0, −14.0, −9.0, −117.5989454]

rho_lbl := ["Planck", "Nuclear", "Water", "CMB", "Lambda_obs", "Graviton ZPE"]


Cosmological Constant Problem: Energy Density Hierarchy
log10(rho [J/m^3])
Planck
Nuclear
Water
CMB
Lambda_obs
Graviton ZPE
100
50
0
−50
−100
log10(rho) [J/m^3]
0
1
2
3
4
5
6
7
System
Planck
Nuclear
Water
CMB
Lambda_obs
Graviton ZPE

"Plot 10 done."



"=="

" FINAL SUMMARY TABLE"

"=="

Quantity	Value	Unit
R	3.922800e+22	m
c	2.997925e+08	m/s
hbar	1.054570e-34	J*s
k_1	3.222223e-23	1/m
omega_1	9.659980e-15	rad/s
T_1	6.504346e+14	s
E_1	1.018713e-48	J
m_g	1.133471e-65	kg
c_2	1.068981e+21	

```
m^3/s^2
N (norm.)                                          7.881727e-35      m^
(-3/2)
-------------------------------------------------------------------------
--
<r>                                                1.961400e+22
m
<r^2>                                              6.155344e+44
m^2
Delta_r                                            1.519294e+22
m
Delta_r/R                                              0.387298

Delta_p                                            1.699030e-57        kg*
m/s
-------------------------------------------------------------------------
--
Delta_r*Delta_p                                    2.581325e-35
J*s
hbar/2                                             5.272850e-35
J*s
Ratio pos-mom                                          0.489550

Delta_E                                            5.093563e-49
J
Delta_t                                            1.035199e+14
s
Delta_t [Myr]                                          3.280098
Myr
Delta_E*Delta_t                                    5.272850e-35
J*s
Ratio E-t                                              1.000000

-------------------------------------------------------------------------
--
sigma_r                                            2.194469e+22
m
sigma_r/R                                              0.559414

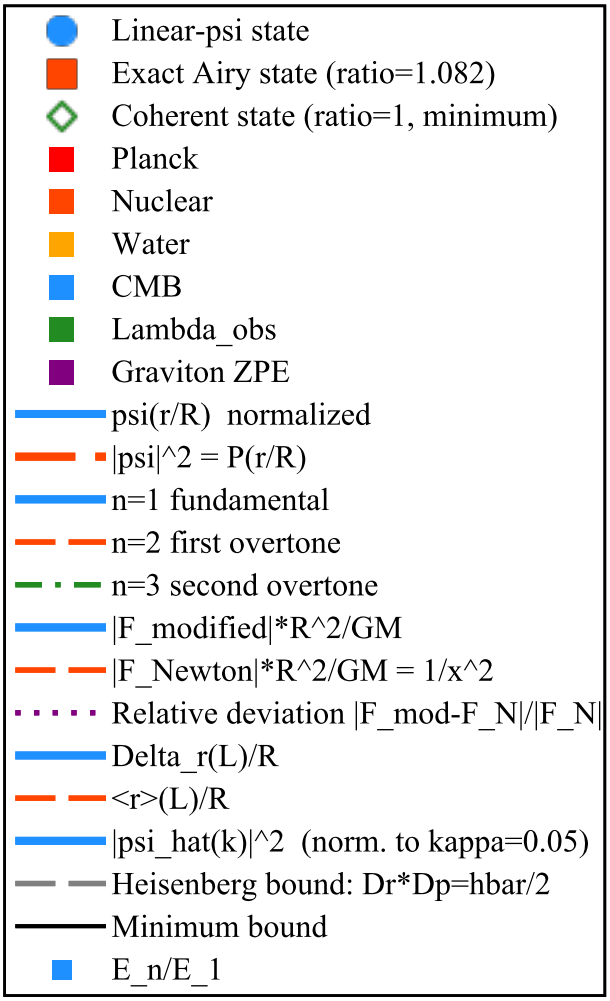
delta_r_min                                        3.103448e+22
m
Delta_F                                            2.351622e-02
N/kg
rho_grav_vac                                      2.517993e-118
J/m^3
rho_Lambda_obs                                     6.000000e-10
J/m^3
Discrepancy                                       2.382850e+108
```

```
----------------------------------------------------------------------
--

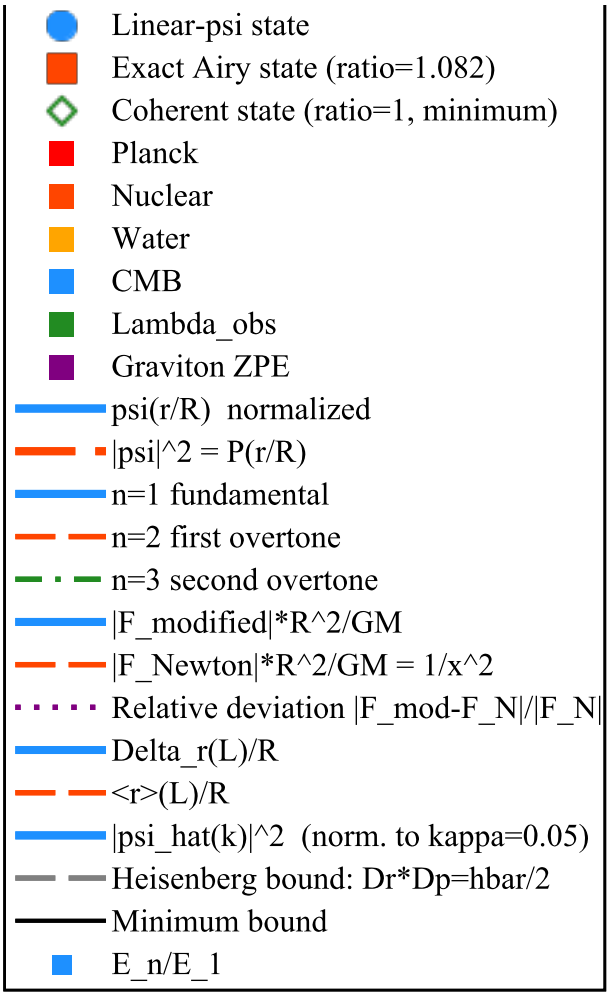
"============================================================="

" COMBINED DISPLAY: 2D Plots (Plots 1,2,3,4,5,8,9,10)"

"============================================================="
```

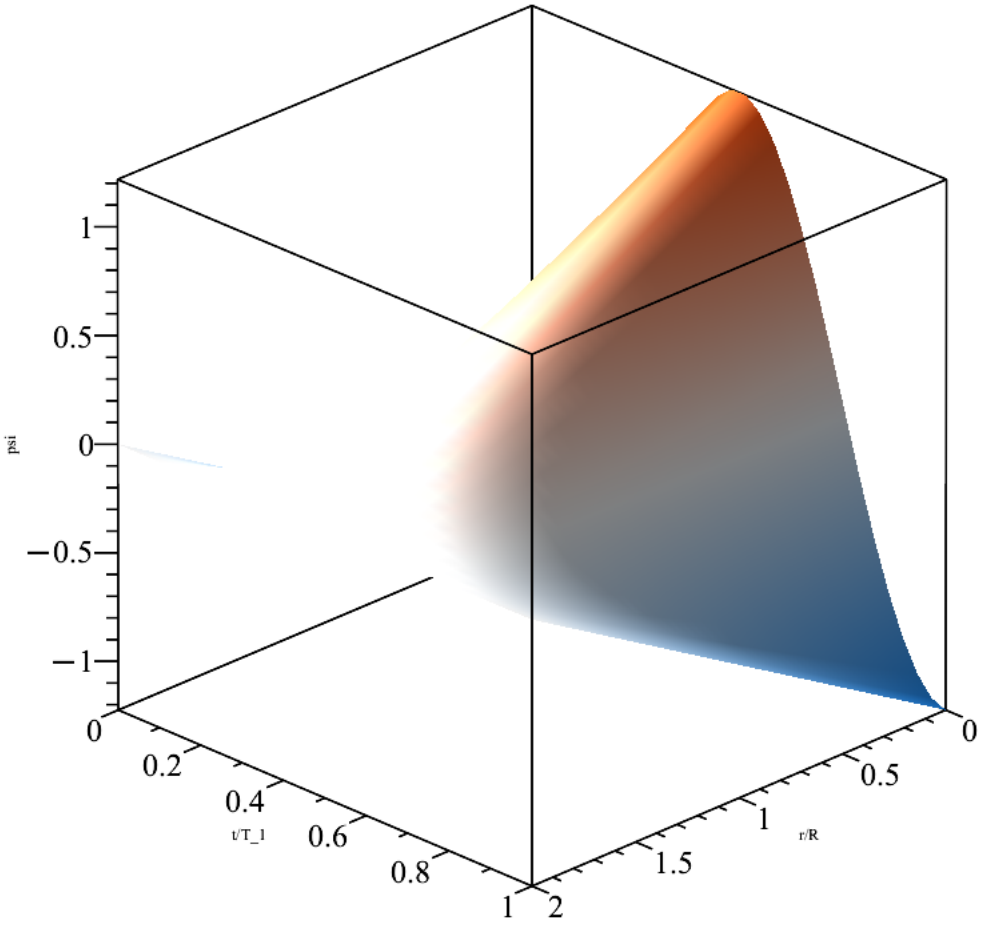


"Combined display ready (insequence=true for slideshow)."

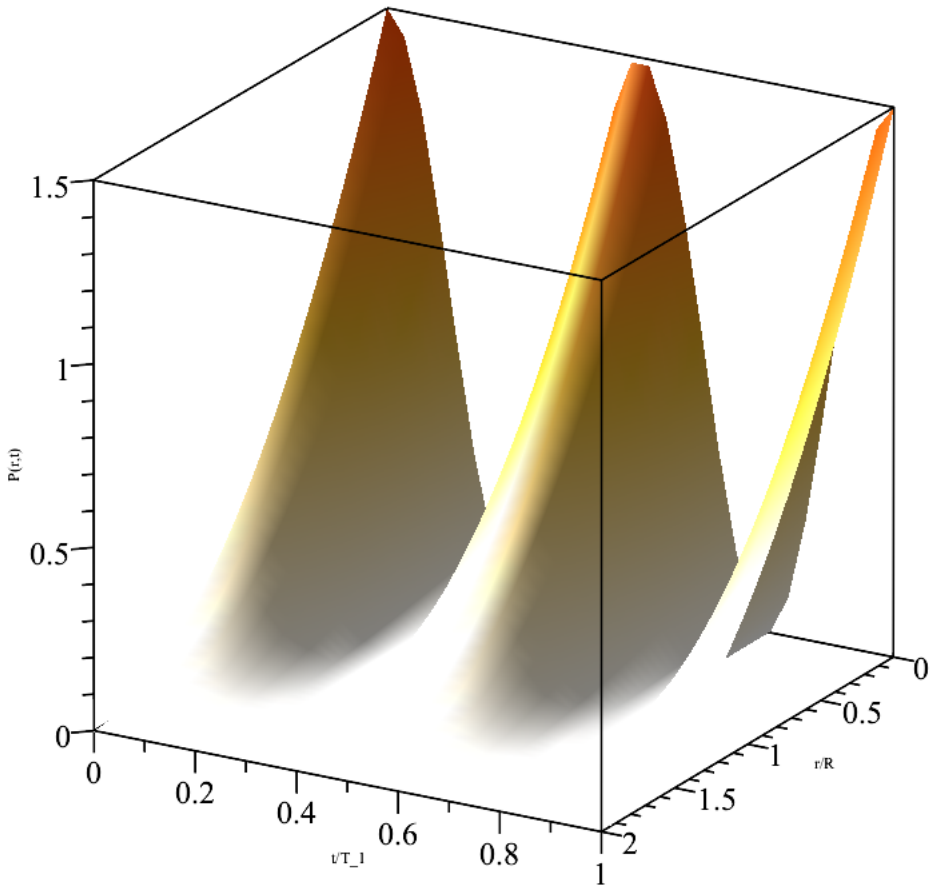


"3D Plots (display individually):"

Graviton Wave Function psi(r,t)=N*(r-2R)*cos(omega1*t)
x=r/R in [0,2], tau=t/T_1



Graviton Probability Density P(r,t)=|psi|^2
P=(3/8)*(r/R-2)^2*cos^2(omega_1*t)



"==========="

" TRANSCRIPT COMPLETE — 10 plots, 0 errors"

"==========="

(1)